FloodRelief

Disaster Response & Resource Coordination Platform

Complete Project Context & Technical Reference

Concept, features, architecture, database, API, security,
testing, deployment and development roadmap

Selected stack

PostgreSQL · Express.js · React · Node.js · TypeScript

Version 1.0 · November 2026
Based on the original FloodRelief Project Context Guide, updated with the decision to use the PERN stack (PostgreSQL, Express.js, React, Node.js) with TypeScript.

Purpose: primary reference for development and for handing the project to another developer or AI assistant. Includes a ready-to-copy continuation prompt.

Demonstration / portfolio project. All data is synthetic. Not an official emergency service.

Table of Contents

How to Read This Document . . . 6

Where the content comes from . . . 6

How original requirements and recommendations are labelled . . . 6

Document map . . . 7

1. Project Concept . . . 8

1.1 What FloodRelief is . . . 8

1.2 What FloodRelief is NOT . . . 8

1.3 The real-world problem . . . 8

1.4 The central workflow . . . 9

1.5 Project type, scope and audience . . . 9

2. Users and Roles . . . 10

2.1 The five roles . . . 10

2.2 Permission matrix . . . 11

2.3 Why backend authorization is necessary . . . 12

3. Complete Feature Documentation . . . 13

3.1 Core features (MVP foundation) . . . 13

3.2 Additional features that can differentiate FloodRelief . . . 20

3.3 Future features (not part of the MVP) . . . 25

4. The PERN Stack Explained from Zero . . . 26

4.1 PostgreSQL: the database . . . 26

4.2 Express.js: the backend framework . . . 28

4.3 React: the frontend library . . . 30

4.4 Node.js: the runtime . . . 31

4.5 TypeScript . . . 32

5. How the Complete Application Works . . . 33

5.1 The big picture . . . 33

5.2 Beginner vocabulary for the conversation . . . 34

5.3 Status codes you will use . . . 34

5.4 What happens inside the backend . . . 35

5.5 Five practical walk-throughs . . . 36

6. Why PERN Instead of Other Stacks . . . 39

6.1 Summary comparison . . . 39
6.2 Stack by stack . . . 40
6.3 Why PERN is a reasonable balance . . . 43
6.4 When another stack might be a better choice . . . 43

7. Database Architecture . . . 44

7.1 Design principles . . . 44
7.2 Entity relationship diagrams . . . 45
7.3 Status values (proposed) . . . 47
7.4 The eleven core tables . . . 47
7.5 Recommended additional tables . . . 51
7.6 Foreign key summary . . . 53
7.7 Example: the water allocation, step by step . . . 53
7.8 How transactions prevent over-allocation . . . 53
7.9 Migrations . . . 55

8. API Architecture . . . 56

8.1 Conventions . . . 56
8.2 Authentication . . . 56
8.3 Users . . . 57
8.4 Organizations . . . 57
8.5 Assistance requests . . . 58
8.6 Verification and duplicates . . . 58
8.7 Resources . . . 59
8.8 Assignments . . . 59
8.9 Volunteer tasks, teams and proof of delivery . . . 60
8.10 Notifications . . . 60
8.11 Dashboard and administration . . . 60
8.12 Example requests and responses . . . 61
8.13 Common HTTP status codes in this API . . . 61

9. Security Architecture . . . 63

9.1 Where each measure lives . . . 63
9.2 Password hashing: Argon2id or bcrypt . . . 63
9.3 Authentication and secure HTTP-only cookies . . . 64
9.4 Authorization and RBAC . . . 64
9.5 Input validation with Zod . . . 65
9.6 Helmet, rate limiting and CORS . . . 65

9.7 Parameterized SQL queries . . . 66
9.8 Environment variables and secret management . . . 66
9.9 CSRF protection . . . 66
9.10 Audit logs . . . 67
9.11 Secure file uploads . . . 67
9.12 Security checklist . . . 67

10. Performance and User Experience . . . 68

10.1 Frontend . . . 68
10.2 Backend and database . . . 69
10.3 Map performance . . . 70
10.4 Performance checklist . . . 71

11. Testing . . . 72

11.1 Types of tests and recommended tools . . . 72
11.2 Unit test example: status transitions . . . 72
11.3 Integration test example: authorization . . . 73
11.4 Database transaction test: preventing over-allocation . . . 73
11.5 Frontend tests . . . 74
11.6 End-to-end test: the demo scenario . . . 74
11.7 Security-focused tests . . . 74
11.8 What to run and when . . . 74

12. Deployment . . . 75

12.1 Deployment architecture . . . 75
12.2 The three parts . . . 75
12.3 Development versus production environments . . . 76
12.4 Database migrations in deployment . . . 76
12.5 Backups . . . 76
12.6 HTTPS . . . 76
12.7 Basic logging and monitoring . . . 76
12.8 Git and GitHub . . . 77
12.9 CI/CD with GitHub Actions . . . 77
12.10 Docker (use it where it helps) . . . 77

13. Development Roadmap . . . 79

13.1 MVP versus later enhancements . . . 84

14. Recommended Project Structure . . . 85

14.1 Repository layout . . . 85
14.2 Purpose of the important backend folders . . . 86
14.3 The separation of responsibilities . . . 86
14.4 Purpose of the important frontend folders . . . 87

15. Things to Avoid in the Initial Version . . . 88

16. Privacy and Responsible Use . . . 89

16.1 Use synthetic demo data . . . 89
16.2 Do not expose exact household locations publicly . . . 89
16.3 Restrict phone numbers and personal details . . . 89
16.4 Do not present it as an official emergency service . . . 89
16.5 Why human verification is important . . . 90
16.6 Suggested demo disclaimer . . . 90

17. Glossary of Technical Terms . . . 91

18. Project Continuation Prompt for Another AI Assistant . . . 94

18.1 Decisions already made and decisions still open . . . 94
18.2 The prompt (copy everything in the boxes below) . . . 94

How to Read This Document

This document is the primary reference for the **FloodRelief** portfolio project. It is written so that a developer, or another AI assistant, can understand the project, the selected technology stack, the features, the database, the APIs and the development plan **without access to any earlier conversation**.

Where the content comes from

Two sources were combined:

- **The original project documentation PDF** (the "FloodRelief Project Context Guide"). It is the source for the project concept, the users, the main features, the MVP scope, the privacy rules and the "what not to build" guidance.
- **The updated technology decision** made afterwards: the PERN stack (PostgreSQL, Express.js, React, Node.js) with TypeScript on both the frontend and backend. This replaces the Angular + Java Spring Boot stack that the original PDF described. Angular and Spring Boot are **not** used anywhere in this document except in the comparison chapter.

How original requirements and recommendations are labelled

Wherever a section goes beyond the original PDF, it is clearly marked. Two colour-coded boxes are used throughout:

ORIGINAL REQUIREMENT
A requirement or idea that comes from the original project PDF or from your instructions. Treat it as part of the project definition.

RECOMMENDATION (not in the original document)
An addition, design choice or suggestion made while preparing this document. It is a proposal that you may change. Examples: extra database tables, specific library choices, the exact status names.

Document map

Chapters	What they cover
1 to 3	The concept, the users and roles, and complete feature documentation.
4 to 6	The PERN stack explained from zero, how the whole application works, and why PERN was chosen over other stacks.
7 to 8	The proposed database design (with ER diagrams) and the REST API design.
9 to 11	Security, performance and user experience, and testing.
12 to 14	Deployment, the 13-phase development roadmap, and the recommended folder structure.
15 to 16	What to avoid in the first version, and privacy and responsible use.
17 to 18	Glossary of terms, and the ready-to-copy Project Continuation Prompt for another AI assistant.

BEGINNER TIP
You do not need to read everything in order. If you are new to web development, read chapters 4 and 5 first: they explain every technology and how the pieces talk to each other. Come back to the feature chapter when you start building each phase.

1. Project Concept

1.1 What FloodRelief is

FloodRelief is a **Disaster Response and Resource Coordination Platform**. It helps coordinate assistance requests, verify reported needs, manage available relief resources, allocate those resources, and track volunteer activity.

Its defining idea is a **traceable workflow**. From the moment someone reports a need until the assigned relief task is completed, every step has an owner, a status and a record:

A person reports a need → an authorized verifier reviews it → an organization records what it can supply → a coordinator allocates resources and creates an assignment → a volunteer accepts and completes the task → the system records who did what and when.

In one sentence: **FloodRelief turns verified community needs into assigned, trackable relief tasks.**

1.2 What FloodRelief is NOT

It is not...	Why the distinction matters
A donation website	It does not collect money. It coordinates real needs, real stock and real people.
A flood information website	It does not publish forecasts, water levels or news. Other services do that better.
A map of affected areas	A map is one view of the data. The record of the request and its workflow is the real product.
A social media platform	There are no public feeds, likes or comments. Access to information is restricted by role.
An official emergency service	It is a portfolio prototype. It must never imply that rescue is guaranteed or that it replaces government or humanitarian systems.

1.3 The real-world problem

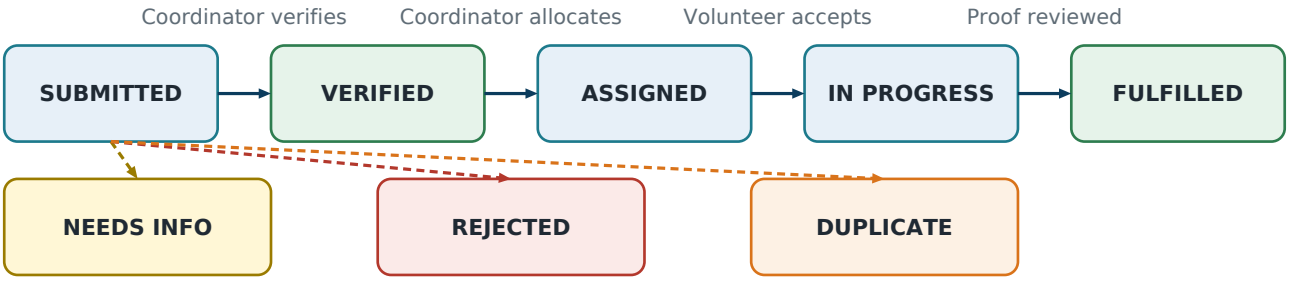
During a flood, people may urgently need drinking water, dry food, medicine, temporary shelter, transport or rescue support. At the same time, volunteers and organizations may hold supplies, vehicles, shelters or willing people. The difficult part is **coordination**, not generosity. Information arrives through phone calls, social media posts, messaging groups and spreadsheets, and different groups cannot see what others are already handling.

Problem	What it looks like in practice	How FloodRelief responds
Scattered information	Requests and available supplies are spread across chats, calls and spreadsheets.	One shared system of record with structured fields.
Duplicate requests	The same household is reported by neighbours, relatives and volunteers.	Duplicate suggestions for a human verifier to confirm or dismiss.
Unclear priority	Responders cannot tell which request is most urgent.	Urgency level, category, status and area filters; sorted lists and map.

Problem	What it looks like in practice	How FloodRelief responds
Resource mismatch	One place receives too much of an item while another has none.	Per-organization inventory and human-approved allocation across organizations.
No clear ownership	A need is visible but nobody is responsible for it.	Assignments and volunteer tasks with a named responsible party.
Weak status visibility	Nobody knows if help is pending, on the way or delivered.	Status workflow, status history and in-app notifications.
Over-promising stock	Two coordinators promise the same 60 bottles of water.	Database transactions that make double allocation impossible.

IMPORTANT LIMITATION (from the original document)
Software cannot create supplies, guarantee safe access, verify every report automatically, or replace trained emergency responders. The platform supports coordination; people and authorized organizations remain responsible for decisions and real-world operations.

1.4 The central workflow



The requester can answer "needs info" and the request returns to SUBMITTED.
Rejected and duplicate requests keep their history; nothing is erased.
Any status may also move to CANCELLED (reporter or coordinator).

Figure 1.1: Proposed life of an assistance request. The exact status names are a recommendation.

The request moves through statuses controlled by the backend. A reporter cannot mark their own request verified, and a volunteer cannot create an allocation. Each change is written to `status_history`.

1.5 Project type, scope and audience

- **Type:** full-stack web application built as a portfolio and learning project.
- **First version:** a controlled prototype that uses **synthetic (fake) data**.
- **Success measure:** a small, complete, well-tested workflow beats a large unfinished feature list.

ORIGINAL REQUIREMENT
Demo scenario from the original PDF: a community reporter submits an urgent clean-water request; a verifier reviews it; an organization records available water supplies; a coordinator allocates supplies and assigns a volunteer; the volunteer updates the task to completed; the dashboard and history reflect the result. Every later chapter supports this scenario.

2. Users and Roles

FloodRelief has five types of users. Each account has a **role**, and the role decides which parts of the system the person may use. The original PDF lists the same five types and notes that the roles can be simplified in the first release (for example, the administrator can also act as verifier).

2.1 The five roles

Role 1: Community reporter / affected person

Who	A person in an affected area, or someone reporting on behalf of a neighbour or relative.
Can do	Register and log in; submit assistance requests; view **only their own** requests and the status history of those requests; answer "needs more information" questions; cancel their own pending request; receive notifications about their requests.
Cannot do	See other people's requests, exact locations or contact details; verify or reject anything; see inventory or allocations; assign tasks.
Why	Reporters are the source of need. They must be able to ask for help easily while their personal information stays protected.

Role 2: Volunteer

Who	An individual who is willing to deliver help, possibly as part of a team or an organization.
Can do	View tasks assigned to them (or offered to their team); accept or decline a task; update progress (accepted, in progress, completed, unable to complete); submit proof of delivery; view the limited details needed to do the task.
Cannot do	Browse all requests; see contact details of tasks that are not theirs; change inventory; verify requests; assign tasks to others.
Why	Volunteers execute the work. They should see only what is necessary for their task (data minimization).

Role 3: Relief organization (organization member)

Who	Staff of an NGO, community group or other relief organization registered on the platform. Inside an organization a member can be an owner, manager or ordinary member (`organization_members.membership_role`).
Can do	Manage **their own** organization profile; create and update **their own** resources and quantities; view allocations that use their resources; manage their own volunteers and teams; view assignments that belong to their organization.
Cannot do	Edit another organization's inventory; verify requests (unless also given the coordinator role); see data outside what their assignments require.
Why	Organizations know what stock they really hold. Letting them keep their own inventory accurate is the basis of trustworthy allocation.

Role 4: Verifier / coordinator

Who	A trained person (for example a union-level relief officer or an experienced volunteer lead) who reviews reports and coordinates the response.
Can do	See all requests in their permitted area; verify, reject or ask for more information; flag or confirm duplicates; set or correct priority; allocate resources from one or more organizations to a verified request; create assignments and volunteer tasks; review proof of delivery; use the dashboard.
Cannot do	Manage platform-wide settings or user accounts (administrator only); silently delete history.
Why	Human judgement is the safety layer. Verification and allocation are never automatic in the MVP.

Role 5: Platform administrator

Who	The person who operates the platform.
Can do	Create, suspend or change roles of accounts; approve organization registrations; manage reference data such as the location hierarchy; read audit logs; act as verifier in the simplified MVP.
Cannot do	Erase audit logs or status history through the normal application. Admin actions are themselves recorded.
Why	Someone must control access and fix mistakes, but administrator power is the most dangerous, so it is the most strictly logged.

2.2 Permission matrix

The table below is a proposed starting matrix. "Own" means only records that belong to the signed-in user or their organization. "Area" means records inside the geographic area assigned to a coordinator.

Action	Reporter	Volunteer	Org member	Coordinator	Admin
Register / log in	Yes	Yes	Yes	Yes	Yes
Submit a request	Yes	No	No	Yes (on behalf)	Yes
View requests	Own	Via task only	Via assignment only	Area	All
Verify / reject / flag duplicate	No	No	No	Yes	Yes
Manage organization profile	No	No	Own org	No	Yes
Manage resources / inventory	No	No	Own org	View all	Yes
Allocate resources to a request	No	No	No	Yes	Yes
Create assignments / tasks	No	No	Own org (optional)	Yes	Yes
Accept / update a task	No	Own	No	View	View

Action	Reporter	Volunteer	Org member	Coordinator	Admin
Submit proof of delivery	No	Own task	No	No	No
Review proof of delivery	No	No	No	Yes	Yes
View dashboard	No	No	Own org summary	Area	All
Manage users and roles	No	No	No	No	Yes
Read audit logs	No	No	No	No	Yes

RECOMMENDATION (not in the original document)
Keep the MVP simple: one role per user account, with the organization link stored separately in `organization_members`. If a person needs two roles (for example a volunteer who is also a coordinator), start with two accounts or add a roles table later.

2.3 Why backend authorization is necessary

A common beginner mistake is to hide the "Verify" button from users who should not see it and assume the system is now secure. It is not.

The browser belongs to the user. Anyone can open the developer tools, read the JavaScript, copy a request and send it again with different values. A person can also skip the React app entirely and call the API with a tool such as curl or Postman. Therefore **everything the frontend does can be imitated by an attacker**.

Example attack. A volunteer notices that the frontend verifies a request by sending `PATCH /api/requests/42/verification`. The button is hidden for volunteers, but the volunteer sends that request manually. If the backend does not check the role, the request is verified even though the volunteer has no right to do it.

Layer	Role of this layer	Is it a security control?
React frontend	Hide buttons, show the right menus, give friendly messages. Improves usability.	**No.** Convenience only.
Express backend	Authenticate the caller, check the role and ownership, validate the data, apply business rules.	**Yes.** The only trustworthy layer.
PostgreSQL	Constraints, foreign keys and transactions protect data integrity even if code has a bug.	**Yes**, as the last line of defence.

The rule used throughout this project: **every protected endpoint checks who is calling (authentication) and whether that caller may perform this action on this record (authorization), on the server, every time.** The frontend only mirrors those rules to make the interface pleasant.

NOTE
Authorization has two parts: **role checks** ("is this user a coordinator?") and **ownership checks** ("is this the reporter's own request?"). Many real bugs come from forgetting the second part: a reporter changing the number in the URL from /requests/42 to /requests/43 and reading someone else's data.

3. Complete Feature Documentation

This chapter documents every feature in the same nine-part format: what it does, why it is needed, who uses it, the workflow, the stored information, the backend APIs, the database tables, what the frontend displays, and security or performance notes. Endpoint paths and table names are the ones proposed in chapters 7 and 8.

> **ORIGINAL REQUIREMENT**
> The 15 core features and the additional features A to G come from your instructions and the original PDF. The detailed endpoint names, columns and status values inside each feature are recommendations that can be adjusted.

3.1 Core features (MVP foundation)

1. User registration and login

What it does	Lets people create an account and sign in. After login the backend knows who the caller is on every later request.
Why it is needed	Every role-based rule depends on knowing the identity of the caller.
Who uses it	Everyone. Reporters and volunteers self-register; coordinator and administrator accounts are created or promoted by an administrator.
Workflow	1) User opens the register form and submits name, email, password. 2) Backend validates with Zod, hashes the password, inserts the user. 3) User logs in with email and password. 4) Backend verifies the hash and sets a secure HTTP-only cookie. 5) The browser sends the cookie automatically on later requests. 6) Logout clears the cookie.
Information stored	Name, email, optional phone, password hash (never the password), role, preferred language, status, created and last-login times.
Backend APIs	`POST /api/auth/register`, `POST /api/auth/login`, `POST /api/auth/logout`, `GET /api/auth/me`
Database tables	`users` (and `audit_logs` for login events)
Frontend shows	Register and login forms with clear error messages, a loading state on the button, a language switch, and a redirect to the right home page for the role.
Security / performance	Argon2id or bcrypt hashing, rate limiting on login, a generic error ("email or password is incorrect") so attackers cannot discover which emails exist, HTTP-only cookies, no passwords in logs. Self-registration must never allow choosing the coordinator or administrator role.

2. Role-based access control (RBAC)

What it does	Restricts every action to the roles that are allowed to perform it, and restricts record access by ownership or area.
Why it is needed	Disaster data is sensitive and the actions (verify, allocate) have real-world consequences.
Who uses it	The system itself; administrators manage roles.
Workflow	A request arrives → authentication middleware identifies the user → `requireRole(...)` middleware compares the role with the allowed list → the service performs an ownership or area check → the action proceeds or the API returns 403.
Information stored	The role on the user record, membership roles in organizations, optionally the coordinator's assigned area.
Backend APIs	Applied to every protected endpoint. Role changes: `PATCH /api/users/:id/role`.
Database tables	`users`, `organization_members`, `audit_logs`
Frontend shows	Menus and buttons appear only for permitted actions. A friendly "you do not have access" page for 403 responses.
Security / performance	The frontend only mirrors the rules; the backend enforces them. Role changes are written to the audit log. Test unauthorized access explicitly (chapter 11).

3. Assistance request submission

What it does	A reporter describes the help needed: category, description, approximate quantity, urgency, location and optional contact details.
Why it is needed	It is the starting point of the whole workflow.
Who uses it	Reporters (and coordinators entering a request on behalf of a caller).
Workflow	1) Reporter fills the form and picks the location from the area hierarchy and/or the map. 2) Frontend checks basic input. 3) Backend validates again, stores the request with status *submitted*, writes the first history row, runs duplicate suggestions, and notifies coordinators of the area. 4) Reporter sees a confirmation with a reference number.
Information stored	Category (water, food, medicine, shelter, transport, rescue, other), description, quantity and unit, urgency (low, medium, high, critical), administrative area, approximate latitude and longitude, optional contact phone (restricted), status, timestamps.
Backend APIs	`POST /api/requests`
Database tables	`assistance_requests`, `status_history`, `admin_areas`, `notifications`
Frontend shows	A short, accessible form in Bangla or English; helper text; urgent safety notice that this is not an emergency hotline; success page with reference number.
Security / performance	Zod validation of every field, size limits on text, rate limit per user to limit spam, phone number stored but hidden from the public, exact coordinates stored privately and only approximate coordinates exposed.

4. Assistance request listing and detail pages

What it does	Shows requests in a filterable, paginated list and a detail page with status, history and (for authorized roles) allocations and tasks.
Why it is needed	Coordinators need to see what is waiting; reporters need to see their own request status.
Who uses it	Reporters (own requests), coordinators and administrators (area or all), volunteers and organizations (only through their tasks and assignments).
Workflow	List page calls the API with filters (status, urgency, category, area, text) and a page number. Clicking an item opens the detail page, which loads the request, its history and related assignment data.
Information stored	Nothing new is stored; this feature reads `assistance_requests`, `status_history`, `assignments`.
Backend APIs	`GET /api/requests`, `GET /api/requests/:id`, `GET /api/requests/:id/history`
Database tables	`assistance_requests`, `status_history`, `assignments`, `users`
Frontend shows	Table with status and urgency badges, filters, sort, pagination controls, skeleton loading rows, empty state ("No requests match these filters"), and a timeline of status changes on the detail page.
Security / performance	Server-side pagination (never load all rows), indexes on status, urgency, area and created_at, field-level filtering so reporters never see other people's phone numbers. Ownership check on `GET /api/requests/:id`.

5. Request verification and rejection

What it does	A coordinator reviews a submitted request and decides: verify, reject, or ask for more information.
Why it is needed	Human verification protects limited resources from fake, mistaken or malicious reports.
Who uses it	Coordinators and administrators.
Workflow	1) Coordinator opens a submitted request. 2) Checks details and duplicate suggestions. 3) Chooses a decision and writes a note (required for rejection or "needs info"). 4) Backend inside one transaction updates the request status, inserts a `request_reviews` row and a `status_history` row, then creates a notification for the reporter.
Information stored	Decision, reviewer, notes, timestamp, previous and new status.
Backend APIs	`PATCH /api/requests/:id/verification`
Database tables	`request_reviews`, `assistance_requests`, `status_history`, `notifications`
Frontend shows	A review panel with the request details, duplicate suggestions, decision buttons, a required note field, and a confirmation step for reject.
Security / performance	Only coordinator or administrator roles; only valid transitions (for example, a fulfilled request cannot be rejected); transaction so the status and the review record never disagree; every decision is auditable.

6. Duplicate request flagging

What it does	Lets a coordinator mark a request as a possible or confirmed duplicate of another, and shows system suggestions (see feature B).
Why it is needed	Duplicates waste resources and make the dashboard misleading.
Who uses it	Coordinators and administrators.
Workflow	On submission the backend looks for nearby similar requests and stores suggestions. A coordinator sees them on the review panel and can *confirm* (status becomes duplicate, linked to the original) or *dismiss*. The system never decides alone.
Information stored	Request A, possible duplicate request B, score or reasons, status (suggested, confirmed, dismissed), decided_by, decided_at.
Backend APIs	`GET /api/requests/:id/duplicates`, `POST /api/requests/:id/duplicate-flags`, `PATCH /api/duplicate-flags/:id`
Database tables	`request_duplicate_flags` (recommended addition), `assistance_requests`
Frontend shows	A side-by-side comparison of two requests with distance, time difference and description, with "Same need" and "Different need" buttons.
Security / performance	The query must be limited by area, category and time window so it stays fast. Only coordinators can see the linked request details.

7. Organization registration and management

What it does	Relief organizations register, are approved, and manage their profile and members.
Why it is needed	Resources and assignments belong to organizations, so organizations must exist as accountable entities.
Who uses it	Organization members register; administrators approve; members manage the profile.
Workflow	1) A user creates an organization (status pending). 2) An administrator reviews and approves. 3) The creator becomes the owner and can add members. 4) Only approved organizations can hold allocatable resources.
Information stored	Name, type (NGO, community group, government unit, other), contact details, area, verification status, members and their membership role.
Backend APIs	`POST /api/organizations`, `GET /api/organizations`, `GET/PATCH /api/organizations/:id`, `PATCH /api/organizations/:id/verification`, members endpoints
Database tables	`organizations`, `organization_members`
Frontend shows	Organization profile page, member list, a banner that says "pending approval" when unverified.
Security / performance	Only owners or managers edit their organization; only administrators change verification status; an unverified organization cannot be used for allocation.

8. Resource and inventory management

What it does	Organizations record what they hold: quantity, unit, location and availability.
Why it is needed	Allocation is only trustworthy if the numbers are accurate and traceable.
Who uses it	Organization members (own stock); coordinators and administrators (read all).
Workflow	A member creates a resource (for example 60 bottles of drinking water), later adjusts the stock with a reason. Every change inserts a row into the inventory ledger.
Information stored	Organization, resource type, unit, total quantity, available quantity, location, status, last-updated time; ledger of movements.
Backend APIs	`POST /api/resources`, `GET /api/resources`, `PATCH /api/resources/:id`, `POST /api/resources/:id/adjustments`, `GET /api/resources/:id/movements`
Database tables	`resources`, `inventory_movements` (recommended addition)
Frontend shows	Inventory table with total and available columns, low-stock highlight, an "adjust stock" dialog that requires a reason, a movement history tab.
Security / performance	Database CHECK constraints (`available_quantity >= 0` and `available_quantity <= total_quantity`); an organization member may touch only their own organization's rows; stock changes must go through service functions that write the ledger.

9. Resource allocation to verified requests

What it does	A coordinator assigns quantities from one or more organizations' resources to a verified request and creates an assignment.
Why it is needed	This is the core of coordination and the place where mistakes are most costly.
Who uses it	Coordinators and administrators.
Workflow	See feature A below for the full workflow and the transaction that prevents over-allocation.
Information stored	Assignment, items (resource and quantity), who created it, time, status.
Backend APIs	`POST /api/assignments`, `GET /api/assignments/:id`, `POST /api/assignments/:id/cancel`
Database tables	`assignments`, `assignment_items`, `resources`, `inventory_movements`, `status_history`
Frontend shows	A two-panel view: the request needs on the left, matching resources by organization on the right, with live remaining quantity and a review step before confirmation.
Security / performance	One database transaction with row locking; the request must be *verified*; no automatic allocation in the MVP.

10. Volunteer and team management

What it does	Volunteers register, belong to organizations or teams, and are visible to coordinators for assignment.
Why it is needed	Tasks need people. Teams make it possible to send a group (for example a boat crew) on one task.
Who uses it	Volunteers, organization managers, coordinators.
Workflow	A volunteer registers and optionally joins an organization. A manager groups volunteers into teams. Coordinators can see availability.
Information stored	Volunteer profile (name, phone restricted, languages, area, availability), team name, team members.
Backend APIs	`GET /api/users?role=volunteer`, `POST /api/teams`, `POST /api/teams/:id/members`
Database tables	`users`, `organization_members`, `teams` and `team_members` (recommended additions)
Frontend shows	Volunteer list with filters (area, language, availability) and a team page.
Security / performance	Volunteer phone numbers are visible only to coordinators when needed. Teams are optional in the first version.

11. Volunteer task assignment

What it does	A coordinator (or authorized organization manager) turns an assignment into one or more tasks for specific volunteers or teams.
Why it is needed	It creates clear ownership of the delivery.
Who uses it	Coordinators, organization managers.
Workflow	From the assignment page the coordinator selects a volunteer, sets a due time and instructions. The volunteer receives an in-app notification.
Information stored	Assignment, volunteer (or team), status, due time, instructions, accepted and completed times.
Backend APIs	`POST /api/assignments/:id/tasks`
Database tables	`volunteer_tasks`, `notifications`, `status_history`
Frontend shows	A "Create task" dialog showing volunteers sorted by area and a summary of what must be delivered.
Security / performance	Only authorized roles create tasks; volunteers only see tasks assigned to them; the task page shows an approximate location until the volunteer accepts.

12. Task acceptance and progress updates

What it does	Volunteers accept or decline a task and move it through its statuses.
Why it is needed	Coordinators need to know whether help is really on the way.
Who uses it	Volunteers; coordinators watch.
Workflow	assigned → accepted → in progress → completed, or unable to complete (with a reason). Each transition is validated by the backend and recorded in the status history.
Information stored	Task status, timestamps, optional reason or note.
Backend APIs	`GET /api/volunteer/tasks`, `GET /api/volunteer/tasks/:id`, `PATCH /api/volunteer/tasks/:id`
Database tables	`volunteer_tasks`, `status_history`, `notifications`
Frontend shows	Mobile-friendly task cards with large buttons for Accept, Start and Complete; clear error text when a transition is not allowed.
Security / performance	A volunteer can only update their own tasks; invalid transitions return 409 or 422; if all tasks are completed the assignment is moved forward by the service.

13. Status history and audit logs

What it does	Records every important status change and every sensitive action, with who and when.
Why it is needed	Accountability, debugging, and trust. The original PDF requires an audit history for verification, allocation and status changes.
Who uses it	Coordinators (status timelines), administrators (audit logs).
Workflow	Services write a history row in the same transaction as the change. Security events (login failures, role changes) go to the audit log.
Information stored	Entity type and id, old and new status, actor, time, optional reason. Audit logs: action, target, actor, IP address.
Backend APIs	`GET /api/requests/:id/history`, `GET /api/admin/audit-logs`
Database tables	`status_history`, `audit_logs` (recommended addition)
Frontend shows	A readable timeline ("Coordinator Rahim verified this request, 14:20") and an admin audit table with filters.
Security / performance	History rows are insert-only: the application provides no update or delete path. Avoid logging passwords, tokens or full phone numbers.

14. Dashboard and basic reporting

What it does	Summaries such as submitted, verified, pending allocation and completed counts, available resources, and activity by area.
Why it is needed	Coordinators need an overview to decide what to do next.
Who uses it	Coordinators, administrators, organization members (own summary).
Workflow	The dashboard calls summary endpoints that run grouped SQL queries on live data.
Information stored	Nothing new; calculated from existing tables.
Backend APIs	`GET /api/dashboard/summary`, `GET /api/dashboard/by-area`, `GET /api/dashboard/resources`
Database tables	`assistance_requests`, `assignments`, `resources`, `volunteer_tasks`
Frontend shows	Count cards, a status breakdown chart, a table by area, and a clear label that demo data is synthetic.
Security / performance	Aggregate in SQL, not in JavaScript; add indexes on grouped columns. Numbers must come from real records and be labelled; never present simulated numbers as real disaster statistics (original PDF).

15. In-app notifications

What it does	Shows messages inside the application when something relevant happens.
Why it is needed	People should learn about verification, assignments and task changes without checking everything manually.
Who uses it	All roles.
Workflow	Services insert notification rows after important events. The frontend polls the unread-count endpoint every 30 to 60 seconds and shows a bell icon badge.
Information stored	Recipient, title, message, link to the related record, read time, created time.
Backend APIs	`GET /api/notifications`, `GET /api/notifications/unread-count`, `PATCH /api/notifications/:id/read`, `POST /api/notifications/read-all`
Database tables	`notifications`
Frontend shows	A bell with an unread badge, a dropdown list, and a "mark all as read" action.
Security / performance	A user can only read their own notifications. Simple polling is enough for the MVP; WebSockets or SMS are later possibilities. Keep messages free of sensitive details.

3.2 Additional features that can differentiate FloodRelief

> **NOTE**
> These seven features come from your instructions. They are what makes FloodRelief more than a basic CRUD application. Each can be built incrementally on top of the core workflow.

A. Smart resource allocation (human-approved)

Example. A verified request needs **100 bottles of drinking water**. Organization A has 60 bottles and Organization B has 50 bottles. A coordinator allocates 60 from A and 40 from B. The remaining 10 bottles at B stay available for other requests.

Step	What happens	Database effect
1	Coordinator opens a verified request that needs 100 bottles of water.	No change.
2	The screen lists resources of type "drinking water" with their available quantities: A = 60, B = 50.	Read only.
3	Coordinator enters 60 from A and 40 from B and presses Confirm.	Single transaction begins.
4	Backend locks the two resource rows, checks A has at least 60 and B at least 40.	Row locks held until commit.
5	Backend inserts one assignment and two assignment_items (60 and 40).	New rows.
6	Backend reduces available quantity: A from 60 to 0, B from 50 to 10, and records two ledger rows.	UPDATE resources; INSERT inventory_movements.
7	Request status becomes *assigned*, history rows and notifications are created.	UPDATE and INSERT.
8	Commit. If anything failed, everything rolls back and the coordinator sees a clear message.	All or nothing.

Preventing double allocation. The same physical stock must never be promised twice. This is enforced at three levels: (1) the transaction with `SELECT ... FOR UPDATE` locks the resource rows so two coordinators are processed one after another; (2) the `CHECK (available_quantity >= 0)` constraint makes a negative stock physically impossible; (3) the interface shows live quantities and handles a 409 Conflict response by refreshing the numbers. Chapter 7 explains the transaction in detail.

ORIGINAL REQUIREMENT
No automatic allocation in the MVP. The system may display helpful figures (for example, the remaining need) but a human coordinator must always confirm.

RECOMMENDATION (not in the original document)
Optional later helper: a "suggested split" button that proposes quantities (for example nearest organizations first). It would only pre-fill the form; the coordinator still approves.

B. Duplicate request detection

The same household may be reported several times by different people. The platform can **suggest** possible duplicates using four signals. A human verifier always makes the final decision.

Signal	How it is checked	Notes
Location proximity	Same union or village, or latitude/longitude within a small radius (for example 300 m). Start with the area hierarchy; use PostGIS distance later.	Approximate coordinates may make distances rough.

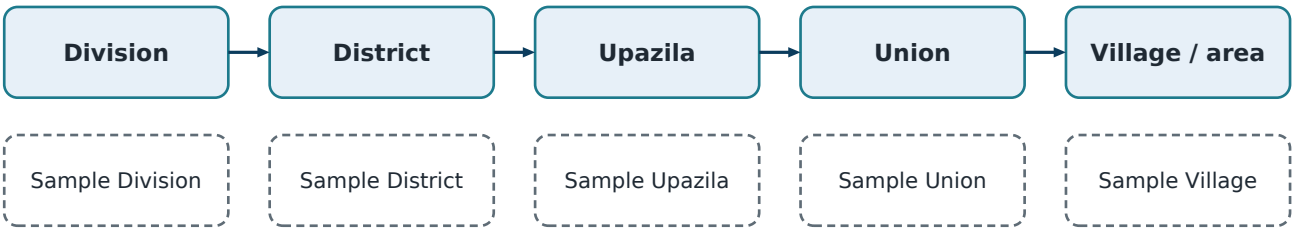
Signal	How it is checked	Notes
Request category	Same category (water, food, medicine...).	Different categories at the same place are usually different needs.
Submission time	Created within a time window (for example the last 48 hours).	Old resolved requests should not be suggested.
Similar descriptions	Simple text similarity. PostgreSQL `pg_trgm` trigram similarity is a lightweight option.	Keep it simple; no AI needed for the MVP.

Suggested flow. When a request is submitted, a service function queries candidate requests with the same category, created within the time window and located in the same area. It scores them and stores suggestions in `request_duplicate_flags` with status *suggested*. The coordinator sees them during review and chooses *confirm* or *dismiss*. A confirmed duplicate is linked to the original request instead of being deleted, so the history remains complete.

> **RECOMMENDATION (not in the original document)**
> Start with a basic rule: same category + same union + last 48 hours + trigram similarity above a threshold. Tune it with synthetic data. More advanced methods belong to the future features list.

C. Bangladesh-specific location hierarchy

Administrative areas in Bangladesh are organized in levels. The platform represents them as a tree so that requests, organizations and coordinators can be filtered and coordinated by area.



Each level stores a parent_id pointing to the level above, so one table (admin_areas) can hold the entire tree. Names shown are placeholders: load real names and codes from an official source when seeding the database.

Figure 3.1: Location hierarchy. Each level points to its parent.

Level	Meaning	Typical use in FloodRelief
Division	Largest administrative region (Bangladesh has 8).	National or regional overview.
District	Subdivision of a division (64 in total).	District-level coordination and reporting.
Upazila	Subdistrict (roughly 490 or more).	Main working level for coordinators.
Union	Smallest rural administrative unit (more than 4,000).	Fine filtering, assigning volunteers who live nearby.
Village / local area	Local place name, optionally ward or neighbourhood.	Describing where help is needed.

Why it is useful. People describe places by these names, not by coordinates. Filtering by upazila or union is faster and more meaningful for coordinators than drawing a map region. It also lets

the system assign a coordinator to a specific area, count requests per area on the dashboard, and offer cascading drop-downs in the request form (division → district → upazila → union).

RECOMMENDATION (not in the original document)
Store the whole tree in one table, `admin_areas`, with `parent_id`, `level`, an English name and a Bangla name. Counts above are approximate and should be verified against an official source when you load real reference data. Place names in this document are placeholders.

D. Interactive map

- **Request markers:** each marker represents a request, colored by urgency. Clicking opens a short summary.
- **Status filtering:** show only submitted, verified or assigned requests.
- **Urgency filtering:** highlight critical and high requests.
- **Location-based search:** jump to an area or place and show nearby requests.
- **Marker clustering:** nearby markers are merged into a numbered circle when zoomed out.
- **Approximate locations:** public or low-privilege views show an approximate position (for example rounded or shifted to the union centre), while exact coordinates stay hidden except for authorized coordinators and assigned volunteers.

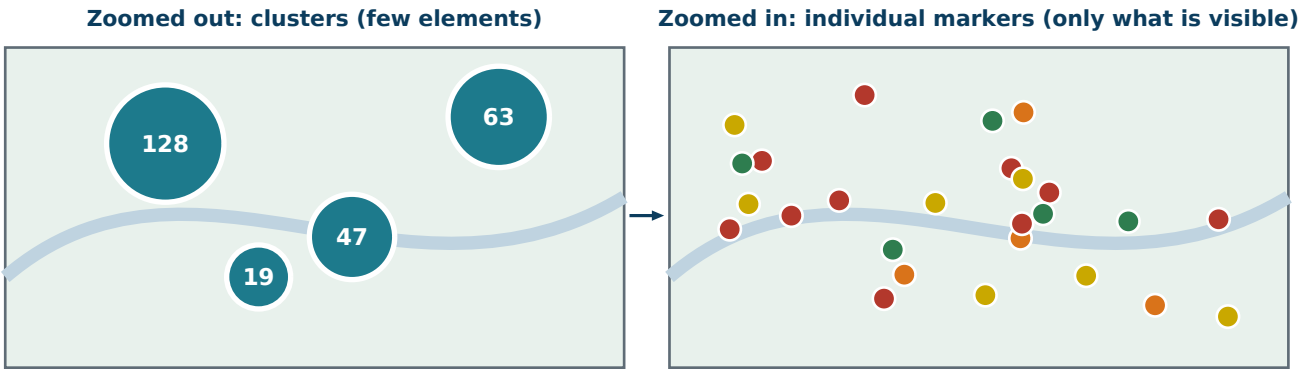


Figure 3.2: Clusters at low zoom, individual markers at high zoom.

Why loading every marker at once is inefficient. If the API returns thousands of requests, the server must query and serialize all of them, the network transfers a large JSON response, and the browser must create thousands of DOM or canvas elements. The map becomes slow, especially on phones with weak connections. Instead the map should request only the markers inside the **visible bounding box** and current filters, clustered on the client (with a library such as Leaflet markercluster) or aggregated on the server when data grows large.

RECOMMENDATION (not in the original document)
Suggested tools: Leaflet with react-leaflet and OpenStreetMap tiles are free and beginner-friendly. Check the tile provider usage policy before deploying publicly.

E. Bangla and English support

Many affected people and local volunteers are more comfortable in Bangla. Bilingual support makes the report form understandable, reduces mistakes, and improves volunteer participation. The original PDF lists multilingual UI as a "later" item, so treat it as an enhancement after the core workflow works.

- Use an internationalization library such as **react-i18next** with two translation files (`en.json`, `bn.json`).
- Never hard-code text in components; always use translation keys, so adding Bangla later is easy.
- Store each user's `preferred_language` and allow switching at any time.
- Store reference data (categories, area names) in both languages (`name_en`, `name_bn`).
- Use a font that renders Bangla properly in the browser and test layout, since Bangla text can be longer or taller than English.
- User-written descriptions stay in the language written; the system does not auto-translate them in the MVP.

F. Proof of delivery

When a volunteer finishes a task, they submit evidence so a coordinator can confirm that the help was actually delivered.

Field	Purpose
Completion time	When the delivery happened (defaults to now, editable).
Delivery status	Delivered fully, delivered partially, could not deliver (with reason).
Optional evidence photo	A photo of the delivered items or handover. Optional because it is not always safe, appropriate or possible.
Notes	Quantity delivered, who received it (role or initials, not unnecessary personal details), problems met.

Coordinator review. The coordinator opens the task, sees the proof, and marks it *accepted* or *needs follow-up*. Only then does the assignment move to completed. Partial deliveries can create a follow-up task.

> **WARNING**
> **Privacy considerations.** Photos can reveal faces, house numbers, children and exact locations. Make photos optional; warn volunteers not to photograph faces or documents; strip image metadata (EXIF, which can contain GPS coordinates) on the server; store files outside the public web root; allow viewing only for coordinators and administrators; delete or archive photos according to a retention rule; and use synthetic images in the demo.

G. Resource availability and accountability

Organizations maintain quantities of what they hold. Instead of overwriting a single number, the platform records an **inventory ledger**: every change is a row in `inventory_movements`.

Movement type	Meaning	Effect on stock
initial	First stock entry.	total and available increase.
adjustment	Correction or new delivery to the organization (with reason).	total and available change.
allocation	Reserved for an assignment.	available decreases; total unchanged.
release	Allocation cancelled or reduced.	available increases.

Movement type	Meaning	Effect on stock
consumed	Delivery confirmed; goods have left the stock.	total decreases; available unchanged.

The current numbers can always be explained by adding up the ledger. This answers the accountability questions: *Who changed this number? When? Why? Which request used it?*

3.3 Future features (not part of the MVP)

The original PDF lists these as later enhancements. They are documented only so that the design does not block them. They are **not** required for the first version.

Future feature	What it could add	When to consider it
SMS notifications	Alerts to people without smartphones or data.	After in-app notifications work; requires an SMS provider, cost and consent handling.
Advanced geographic matching	Find the nearest organization or volunteer using PostGIS distance queries.	When area filters are no longer enough.
Advanced analytics	Trends, response times, area comparisons.	When there is enough real or realistic data to analyze.
Offline-first mobile support	Work with poor connectivity and sync later (for example as a Progressive Web App).	After the responsive web app is stable.
More advanced duplicate detection	Better text similarity, phone-number matching with consent, learned models.	If simple rules produce too many mistakes.
Reporting and export	CSV or PDF exports for coordinators and partners.	After the dashboard is stable; exports need strict access control.

4. The PERN Stack Explained from Zero

ORIGINAL REQUIREMENT
Selected direction: **PERN** = **P**ostgreSQL + **E**xpress.js + **R**eact + **N**ode.js, written in **TypeScript** on both frontend and backend. The original PDF proposed Angular + Spring Boot + PostgreSQL/PostGIS; that choice has been replaced.

A **technology stack** is the set of tools that together make up an application. A web application needs something to show screens, something to apply rules, and something to remember data. PERN names one choice for each job:

Letter	Technology	Job in FloodRelief	Restaurant analogy
P	PostgreSQL	Stores all data permanently.	The pantry and the record book.
E	Express.js	Receives requests and applies the rules.	The kitchen manager who checks every order.
R	React	Builds the screens in the browser.	The menu and the dining room.
N	Node.js	Runs JavaScript/TypeScript on the server so Express can work.	The kitchen itself (the place where work happens).
+	TypeScript	Adds type checking to the code.	A checklist that catches wrong orders before they reach the table.

4.1 PostgreSQL: the database

What a database is and why FloodRelief needs one

A **database** is organized, permanent storage for information. A program's variables disappear when it stops; a database keeps data even when the browser is closed or the server restarts. FloodRelief must remember users, requests, organizations, inventory, assignments and history, so without a database it would lose all operational records.

What "relational" means

A **relational database** stores data in **tables** and lets tables refer to each other. Think of several connected spreadsheets with strict rules.

Term	Meaning	FloodRelief example
Table	A collection of one kind of thing.	`assistance_requests`
Row	One item in a table.	One specific request, for example request number 42.
Column	One property of every row, with a data type.	`urgency`, `created_at`
Primary key (PK)	A column that uniquely identifies each row.	`assistance_requests.id`

Term	Meaning	FloodRelief example
Foreign key (FK)	A column that points to the primary key of another table.	`assistance_requests.reporter_id` points to `users.id`
Relationship	How rows in two tables are linked: one-to-many or many-to-many.	One organization has many resources.

One-to-many: one user submits many requests. **Many-to-many:** users and organizations (a person can belong to several organizations and an organization has many people) are connected through a middle table, here `organization_members`.

SQL

SQL (Structured Query Language) is the language used to ask the database for data or to change it. Four basic operations: SELECT (read), INSERT (add), UPDATE (change), DELETE (remove).

```
-- Create a request (the values are sent separately as parameters, see chapter 9)
INSERT INTO assistance_requests (reporter_id, category, urgency, status)
VALUES ($1, 'water', 'high', 'submitted');

-- Read all verified, critical requests in one upazila, newest first
SELECT id, category, quantity_needed, created_at
FROM assistance_requests
WHERE status = 'verified' AND urgency = 'critical' AND area_id = $1
ORDER BY created_at DESC
LIMIT 20;
```

Constraints

Constraints are rules enforced by the database itself, even if application code has a bug. Examples used in FloodRelief:

- `NOT NULL`: every request must have a category.
- `UNIQUE`: two users cannot register the same email.
- `FOREIGN KEY`: a request cannot point to a user that does not exist.
- `CHECK (available_quantity >= 0)`: stock can never become negative.
- Enumerated values (enum or CHECK): urgency may only be low, medium, high or critical.

Transactions

A **transaction** groups several database steps into one all-or-nothing unit. Either every step succeeds and is committed, or an error happens and everything is rolled back. Allocating 60 bottles from Organization A and 40 from Organization B must not stop halfway; a transaction guarantees that. Transactions are the reason a relational database is recommended for inventory (chapter 7 shows the details).

Indexes

An **index** is like the index at the back of a book. Without it the database reads every row to find matches; with it the database jumps straight to the right place. FloodRelief will index columns that are frequently filtered or sorted, such as request status, urgency, area and creation time. Indexes speed up reads but slightly slow writes and use disk space, so add them for real queries, not for every column.

Why PostgreSQL suits this project

- Strong support for relationships, constraints and **ACID transactions** (reliable, consistent changes).

- Row-level locking (`SELECT ... FOR UPDATE`) for safe inventory allocation.
- Rich data types (timestamps with time zone, enums, JSON when needed) and powerful indexes.
- Free and open source, widely supported by hosting providers.
- Extensions: **PostGIS** for geography and **pg_trgm** for text similarity.

What PostGIS is

PostGIS is an extension that adds geographic data types and spatial functions to PostgreSQL. It is not a separate database. With it you can ask "which verified requests are within 2 km of this point?" or "which requests are inside this upazila boundary?" efficiently, using spatial indexes. FloodRelief does not need PostGIS on day one: store `latitude` and `longitude` first and add PostGIS when distance or boundary queries become real requirements (the original PDF makes the same point).

PostgreSQL compared with MongoDB

Question	PostgreSQL (relational)	MongoDB (document)
Data shape	Tables with fixed columns and links between them.	Flexible JSON-like documents in collections.
Relationships	First-class: foreign keys and joins.	Possible by embedding or referencing, but links are not enforced by default.
Consistency rules	Constraints and transactions are central.	Multi-document transactions exist but the model encourages embedding to avoid them.
Flexible, changing fields	Needs migrations (JSON columns are available).	Very convenient.
FloodRelief fit	Strong: users, organizations, requests, inventory and assignments are tightly related and consistency matters.	Workable but less natural for allocation and accountability.

> **NOTE**
> MongoDB is not a bad database. It is a good choice for flexible, document-shaped data. PostgreSQL is recommended here because FloodRelief's core data is structured and relational and inventory operations need strong consistency.

4.2 Express.js: the backend framework

What a backend is

The **backend** is the part of the application that runs on a server, out of the user's reach. It receives requests from the frontend, checks who the caller is and what they may do, applies business rules, reads and writes the database, and sends back a response. It is the trusted gatekeeper: the frontend must never talk directly to the database.

What a framework does, and what Express does

A **framework** provides a ready-made structure for common tasks. Express is a small, popular web framework for Node.js. It listens for HTTP requests, matches them to **routes**, runs **middleware** functions, and lets you send responses. It is deliberately minimal: it does not force a folder structure, so you must design a clean one yourself (chapter 14).

Routes and controllers

A **route** connects an HTTP method and URL to a function. A **controller** is that function: it reads the request, calls a service, and sends the response. Controllers stay thin.

```
// routes/requests.routes.ts
router.post("/", requireAuth, validate(createRequestSchema), requestsController.create);
router.get("/:id", requireAuth, requestsController.getById);

// controllers/requests.controller.ts
export async function create(req: Request, res: Response) {
  const created = await requestsService.createRequest(req.user!, req.body);
  res.status(201).json(created);
}
```

Middleware

Middleware functions run in order before the controller. Each one can inspect or modify the request, stop it with an error, or call `next()` to continue. Typical middleware in FloodRelief: security headers (Helmet), CORS, JSON body parsing, rate limiting, cookie reading, authentication, role check, input validation, and the final error handler.

```
export function requireRole(...allowed: Role[]) {
  return (req: Request, res: Response, next: NextFunction) => {
    if (!req.user) return res.status(401).json({ error: "Not signed in" });
    if (!allowed.includes(req.user.role))
      return res.status(403).json({ error: "Not allowed" });
    next();
  };
}
// usage: router.patch("/:id/verification", requireAuth,
//          requireRole("coordinator", "admin"), controller.verify);
```

Services and business logic

A **service** contains the rules of the application, independent of HTTP. Examples: "only a submitted request can be verified", "an allocation must not exceed available stock", "write a history row whenever status changes". Keeping rules in services makes them testable without starting a web server and reusable by different controllers.

How Express communicates with PostgreSQL

Node uses a database driver. The most common for PostgreSQL is the **pg** library. The backend creates a **connection pool** (a small set of reusable database connections) and runs **parameterized queries**. A **repository** module holds the SQL so that no other layer writes SQL. Alternatives such as Prisma, Drizzle or Knex exist; see the recommendation box.

```
// db/pool.ts
import { Pool } from "pg";
export const pool = new Pool({ connectionString: process.env.DATABASE_URL });

// repositories/requests.repo.ts
export async function insertRequest(r: NewRequest, reporterId: string) {
  const { rows } = await pool.query(
    `INSERT INTO assistance_requests (reporter_id, category, urgency, description)
     VALUES ($1, $2, $3, $4) RETURNING id, status, created_at`,
```

```
    [reporterId, r.category, r.urgency, r.description]
  );
  return rows[0];
}
```

RECOMMENDATION (not in the original document)
For learning purposes, start with **pg + plain SQL** and a migration tool such as **node-pg-migrate**. Writing SQL yourself teaches relational thinking and makes transactions and locking explicit. A query builder (Knex, Kysely) or ORM (Prisma, Drizzle) can be adopted later if you prefer; keep the choice behind the repository layer so the rest of the code does not change.

How authentication and authorization work in the backend

Authentication answers "who are you?"; **authorization** answers "what may you do?". In FloodRelief: at login the backend checks the password hash and issues a signed session or token stored in an **HTTP-only cookie**. On every later request, an authentication middleware reads that cookie, verifies it, loads the user and attaches it to `req.user`. Then `requireRole` checks the role, and the service checks ownership or area. Details are in chapter 9.

4.3 React: the frontend library

What a frontend is, and what React does

The **frontend** is what the user sees and touches in the browser: pages, forms, tables, the map. **React** is a JavaScript library for building user interfaces from small, reusable pieces called components. FloodRelief will be a **single-page application**: the browser loads the app once, and React changes what is shown without reloading the whole page, fetching data from the API as needed.

Components, props and state

Concept	Meaning	FloodRelief example
Component	A function that returns a piece of interface.	`RequestCard`, `UrgencyBadge`, `Sidebar`
Props	Inputs passed from a parent to a component (read-only).	`<RequestCard request={r} />`
State	Data that belongs to a component and can change over time. When it changes, React re-renders.	The current filter, the form values, whether a dialog is open.

```
type Props = { category: string; urgency: "low" | "medium" | "high" | "critical" };

export function RequestCard({ category, urgency }: Props) {
  return (
    <div className="card">
      <h3>{category}</h3>
      <span className={`badge badge-${urgency}`}>{urgency}</span>
    </div>
  );
}
```

Hooks

Hooks are special functions that let components use features such as state and side effects. The most important: `useState` (remember a value), `useEffect` (run code after rendering, for example to fetch data), `useMemo` and `useCallback` (avoid repeated work), and custom hooks you write yourself (for example `useRequests(filters)`).

Forms

Forms hold user input in state. For FloodRelief, use **React Hook Form** together with **Zod** (the same schema library as the backend, optionally sharing types). This gives instant, friendly validation messages. Remember: frontend validation improves usability only; the backend validates again.

Routing

Routing maps URLs in the browser to pages (`/login`, `/requests`, `/requests/42`). **React Router** is the usual library. Protected routes redirect unauthenticated users to login and hide pages that the role does not need.

API communication

The frontend calls the backend with `fetch` (built into browsers) or **axios**. For loading and caching data, **TanStack Query (React Query)** is strongly recommended: it handles loading states, errors, caching and refetching with very little code.

```
const { data, isLoading, error } = useQuery({
  queryKey: ["requests", filters, page],
  queryFn: () => api.get("/api/requests", { params: { ...filters, page } }),
});
if (isLoading) return <Skeleton />;
if (error) return <ErrorMessage />;
return <RequestTable rows={data.items} />;
```

How React updates the interface

React keeps a lightweight description of the interface in memory. When state or props change, it computes the new description, compares it with the previous one, and updates **only the parts of the real page that changed**. You describe *what the screen should look like for the current data*; React figures out the minimum update.

4.4 Node.js: the runtime

Originally JavaScript could run only inside web browsers, because the browser contained the engine that understood it. **Node.js** takes that same engine (Google's V8) and packages it as a program that runs on a normal computer or server. That is why JavaScript (and TypeScript, after compilation) can run outside the browser.

Node.js does not replace Express: it is the **runtime** that runs your Express code. When you start the backend, Node.js loads your files, opens a network port and waits. When a request arrives, Node calls the right Express handler. Node uses an **event loop** and non-blocking input/output: while waiting for the database to answer one request, it can start handling another. This suits FloodRelief, whose work is mostly waiting for database queries. It is less suitable for heavy CPU calculations, which the project does not need.

Node also brings **npm**, the package manager used to install libraries such as Express, pg, Zod and React itself.

4.5 TypeScript

TypeScript is JavaScript plus a **type system**. You describe the shape of your data, and the compiler warns you about mistakes before the program runs. TypeScript is compiled into plain JavaScript, which is what Node.js and browsers actually run.

```
// JavaScript: nothing stops this mistake until the program fails at run time
function verify(request) { request.status = "verifed"; }   // typo!

// TypeScript: the typo is caught immediately in the editor
type RequestStatus = "submitted" | "needs_info" | "verified" | "rejected" | "duplicate";
interface AssistanceRequest { id: string; status: RequestStatus; urgency: Urgency; }
function verify(r: AssistanceRequest) { r.status = "verifed"; }  // error
```

Why use it in both frontend and backend

- One language to learn for the whole application.
- Share types and Zod schemas between frontend and backend (for example the shape of a request), reducing mismatches.
- Better editor help: autocomplete, safe refactoring, early error detection.
- Valued in job markets and portfolios.

Benefits and limitations for a beginner

Benefits	Limitations
Catches typing mistakes early; self-documenting code.	More to learn at the start (types, generics, configuration).
Excellent tooling and refactoring support.	Error messages can be confusing at first.
Makes large codebases easier to maintain.	Types vanish at run time. They do not validate real incoming data, so Zod validation is still required.
Works with the whole JavaScript ecosystem.	Some libraries have imperfect types.

BEGINNER TIP
Start with simple types (strings, numbers, interfaces, union types like the status above). Turn on strict mode from the beginning, but do not try to learn advanced generics before you need them.

5. How the Complete Application Works

5.1 The big picture

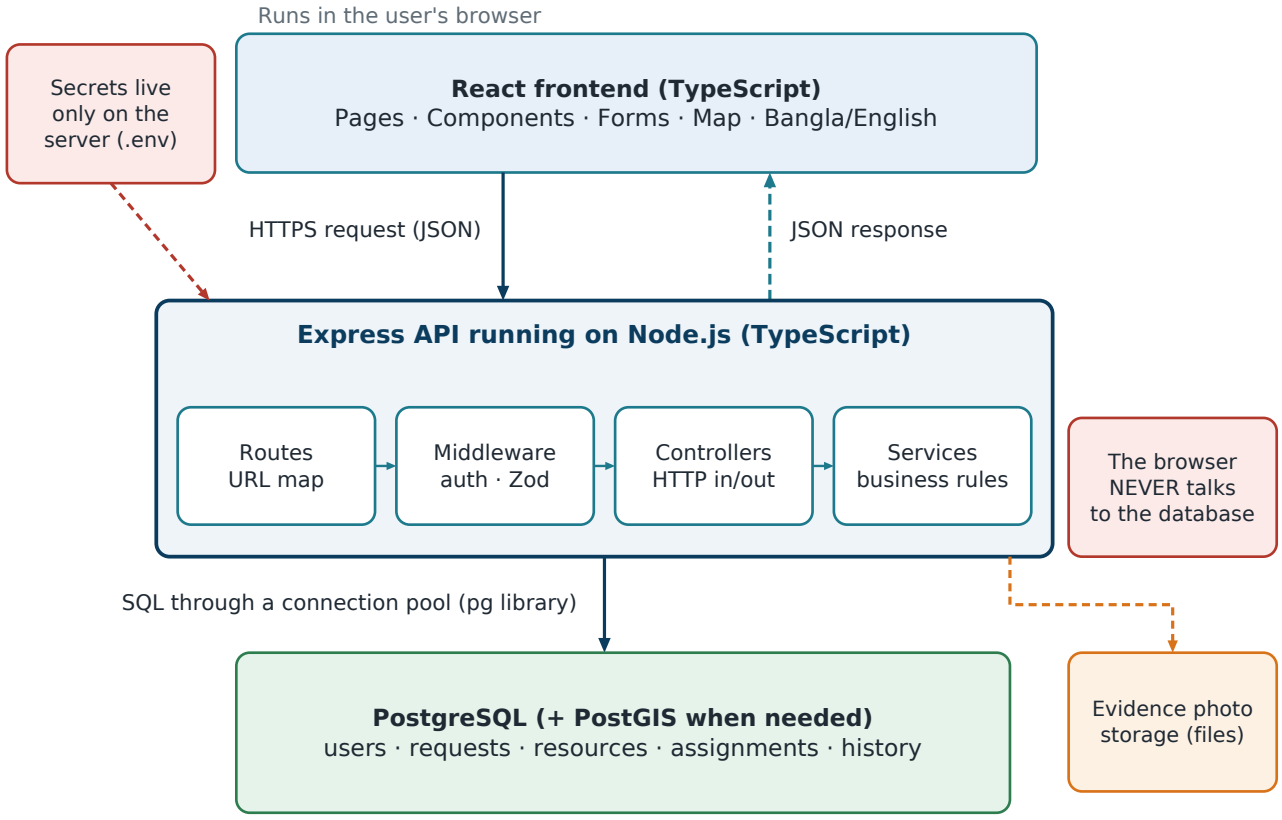


Figure 5.1: High-level architecture. The browser only ever talks to the Express API; only the backend talks to PostgreSQL.

The communication path is: **React frontend → HTTPS request → Express API (running on Node.js) → SQL query → PostgreSQL → result back to the backend → JSON response → React updates the screen.**

5.2 Beginner vocabulary for the conversation

Term	Simple explanation	Example
HTTP request	A message from the browser to the server asking for something.	`POST /api/requests`
Method	The kind of action: GET reads, POST creates, PATCH changes part of something, PUT replaces, DELETE removes.	`PATCH`
Endpoint / URL	The address of an operation on the API.	`/api/requests/42`
Headers	Extra information sent with a request or response (content type, cookies).	`Content-Type: application/json`
JSON	A text format for data, easy for both JavaScript and humans to read.	`{ "category": "water" }`
Status code	A number in the response that tells whether it worked.	`201 Created`
Cookie	A small value the browser stores and sends back automatically.	Session cookie after login.

Example request and response for submitting a need (the values are synthetic):

```
POST /api/requests HTTP/1.1
Content-Type: application/json
Cookie: session=...

{ "category": "water", "urgency": "high", "quantityNeeded": 100, "unit": "bottle",
  "description": "Families on the roof of a school, no clean water since yesterday",
  "areaId": "a3f1...", "lat": 24.8801, "lng": 91.8702 }

HTTP/1.1 201 Created
{ "id": "7c2e...", "reference": "FR-000042", "status": "submitted" }
```

5.3 Status codes you will use

Code	Name	Meaning in FloodRelief
200	OK	Request succeeded and returns data.
201	Created	A new record was created (for example a request or assignment).
204	No Content	Success with nothing to return (for example logout).
400	Bad Request	The request is malformed.
401	Unauthorized	Not signed in or the session is invalid.
403	Forbidden	Signed in but not allowed (wrong role or not the owner).
404	Not Found	The record does not exist (or must not be revealed).
409	Conflict	The action conflicts with the current state, for example not enough stock.
422	Unprocessable Content	Well-formed request whose data failed validation.
429	Too Many Requests	Rate limit exceeded.
500	Internal Server Error	An unexpected bug on the server.

5.4 What happens inside the backend

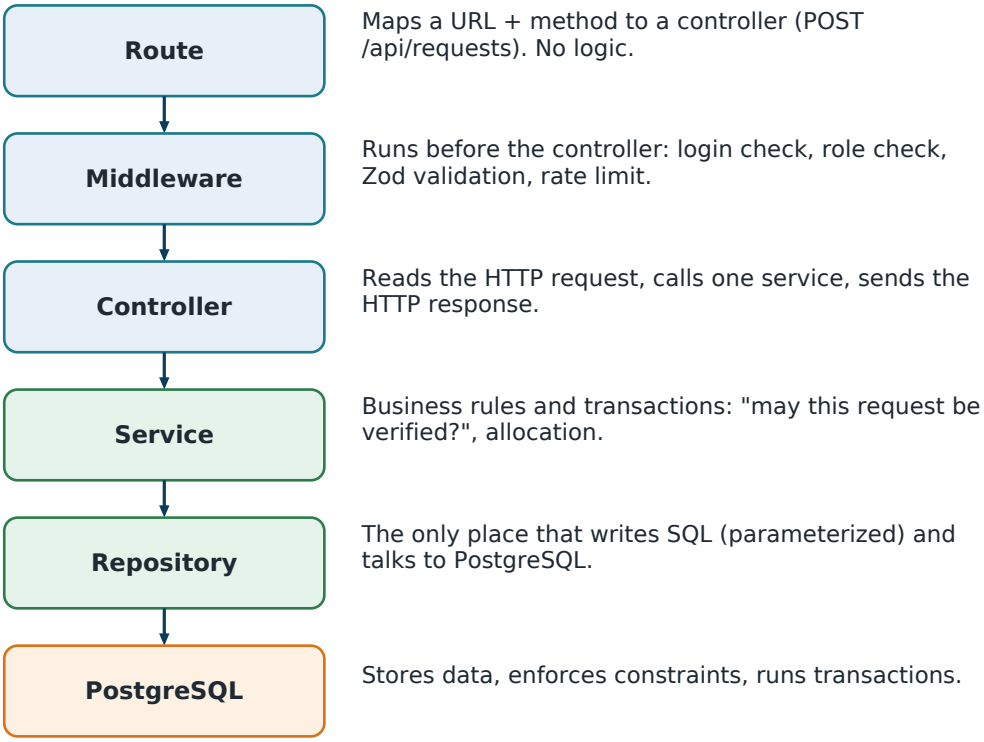
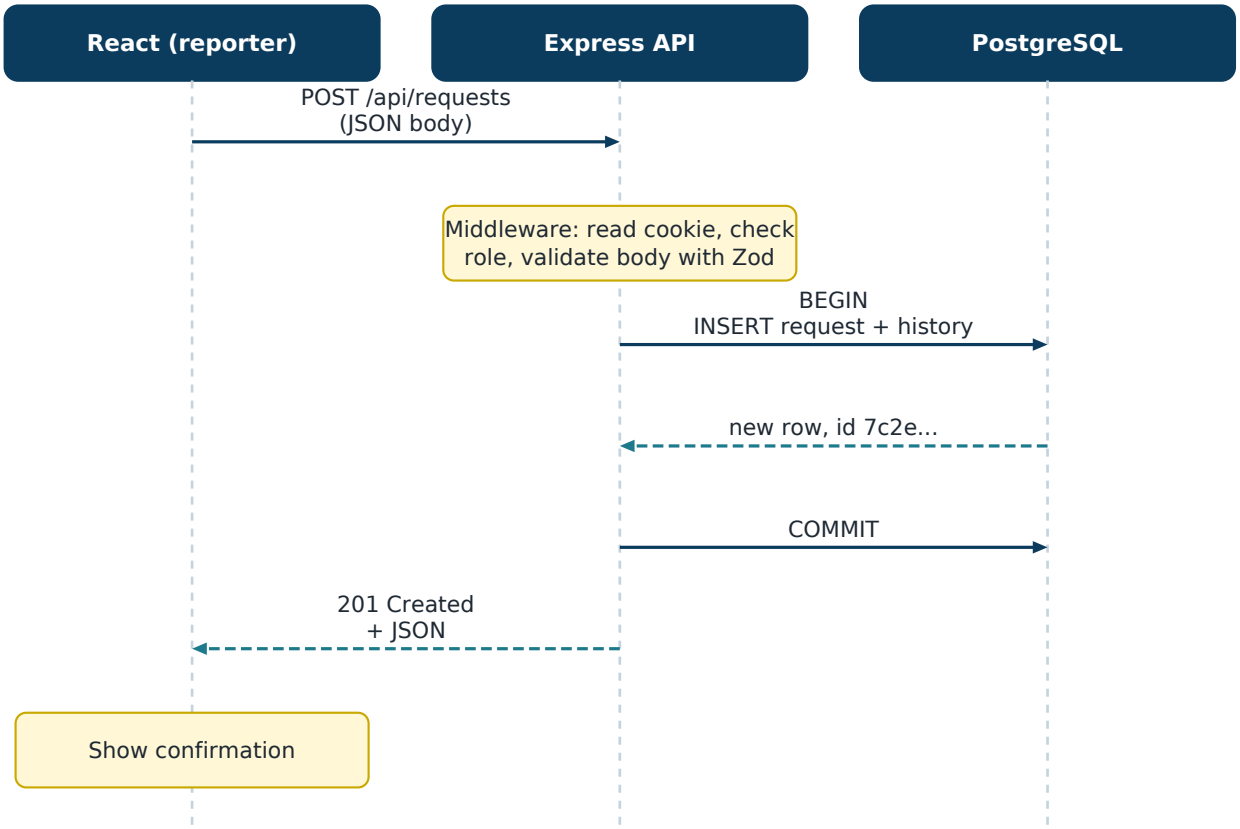


Figure 5.2: The layered backend. Each layer has exactly one job.

Because each layer has one job, you can test rules in the service without HTTP, change SQL in the repository without touching controllers, and add a new middleware (for example logging) without changing business logic.

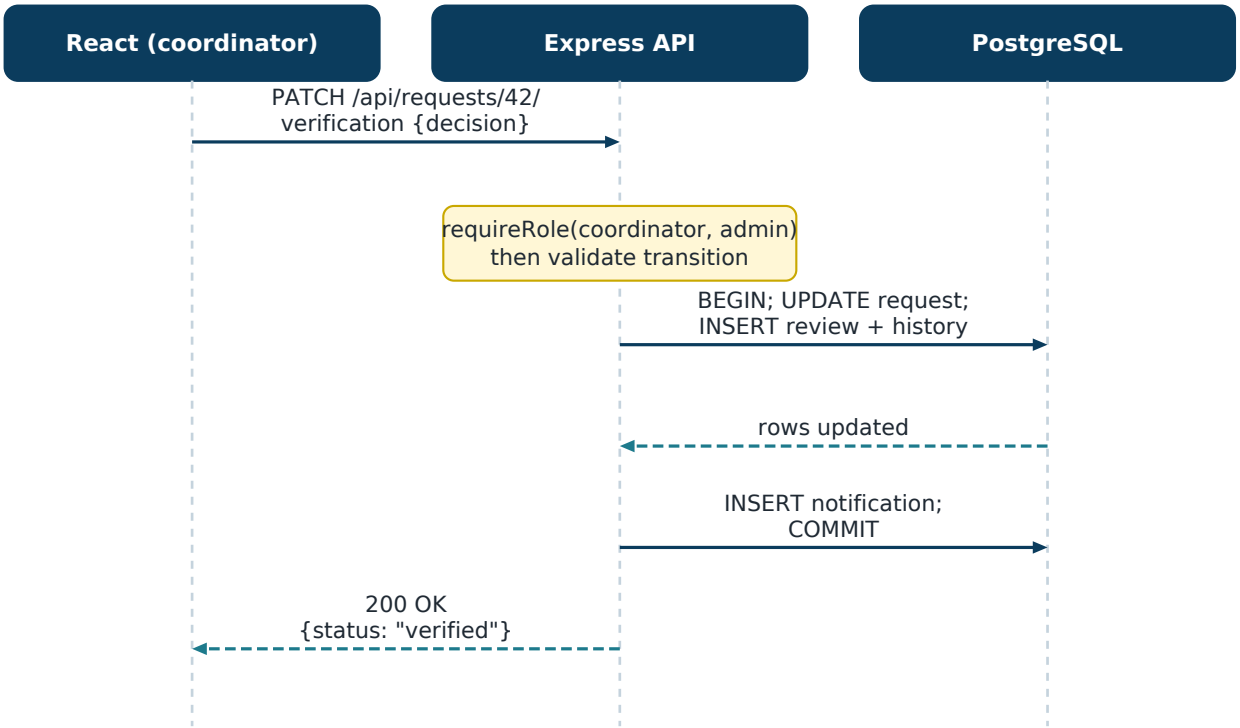
5.5 Five practical walk-throughs

Scenario 1: A reporter submits an assistance request



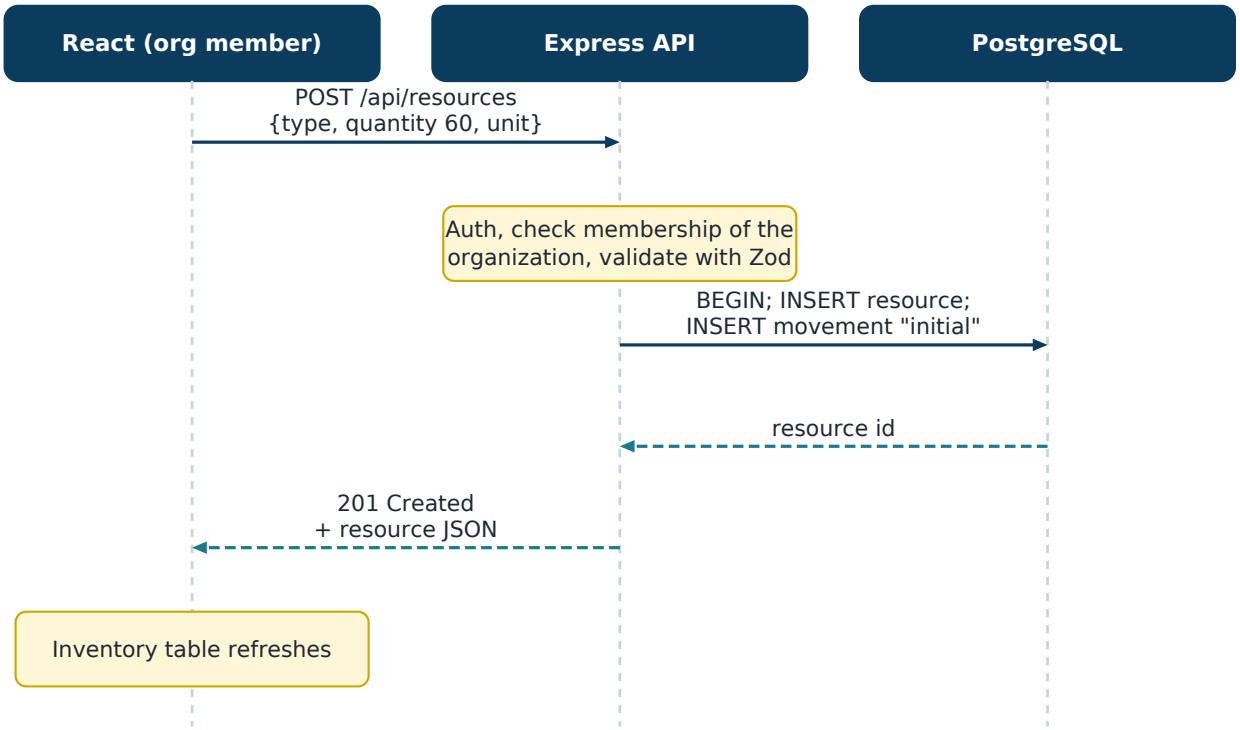
The backend does not trust the form: it checks the cookie, the role, and every field again. The request row and its first history row are written in one transaction. Suggested duplicates and coordinator notifications are created by the service after the insert.

Scenario 2: A verifier approves the request



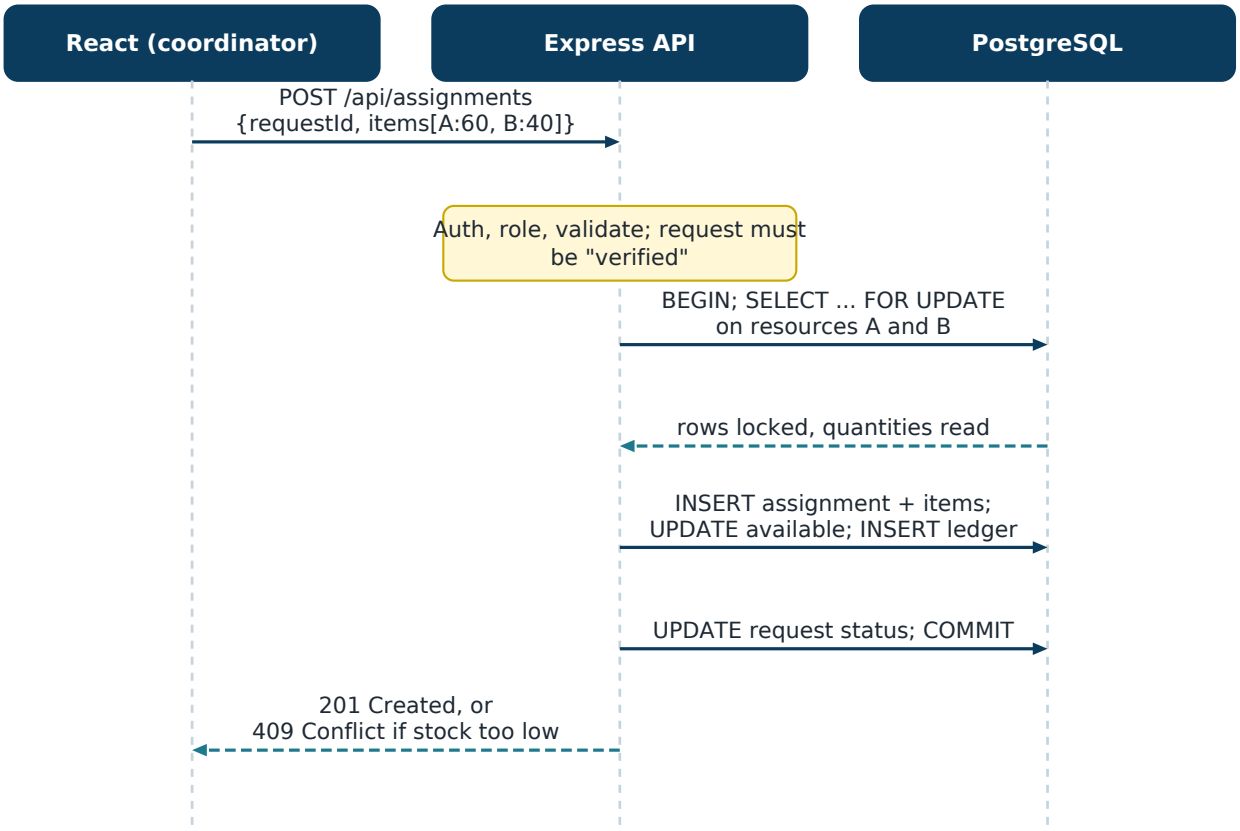
If a volunteer sends the same request, the role middleware stops it with **403 Forbidden** before any database work. If the request was already verified, the service returns **409 Conflict**.

Scenario 3: An organization adds resources



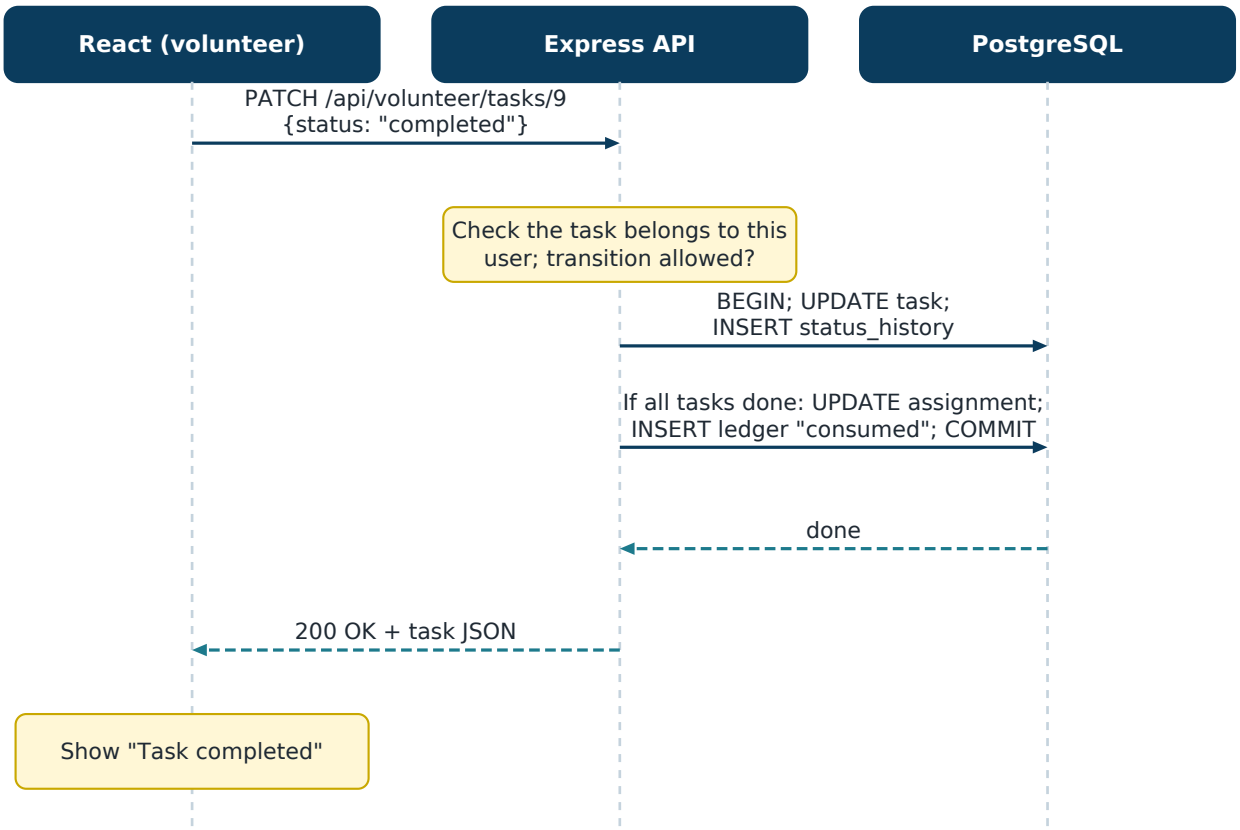
The organization id is taken from the user's membership on the server, **not** from a value the browser sends, so a member cannot add stock to another organization.

Scenario 4: A coordinator allocates resources



This is the most important transaction in the project; it is described step by step in chapter 7.

Scenario 5: A volunteer updates a task to completed



The volunteer never sends the new request status or inventory numbers. The backend derives them from the rules, so a client cannot fake them.

> **RECOMMENDATION (not in the original document)**
> When the stock is "consumed" (physically delivered) is a design decision. This document proposes that the ledger records it when the coordinator accepts the proof of delivery. You can simplify and record it at task completion in the first version.

6. Why PERN Instead of Other Stacks

No stack is universally best. The right choice depends on the goals, the team, the data and the timeline. This chapter compares PERN with six other combinations in the same structure. The comparison is qualitative: no benchmark numbers are claimed.

6.1 Summary comparison

Stack	Learning curve for you	Database fit for FloodRelief	Backend structure	Overall fit
PERN	Low to medium	Excellent	Flexible, you design it	Good balance (selected)
MERN	Low to medium	Fair	Flexible	Good, but weaker for relational data
MEAN	Medium to high	Fair	Flexible	Acceptable, heavier frontend
Angular + Spring Boot + PostgreSQL	High (two ecosystems)	Excellent	Very structured	Strong but heavy to learn
React + NestJS + PostgreSQL	Medium	Excellent	Structured	Very good, natural next step
Next.js + Supabase	Low to medium	Excellent (Postgres)	Largely managed	Fast prototype, less backend learning
Laravel + React	Medium (adds PHP)	Very good	Very structured, batteries included	Strong productivity

6.2 Stack by stack

1. MERN: MongoDB + Express + React + Node

Technologies	MongoDB (document database), Express, React, Node.js. Typically JavaScript or TypeScript throughout.
Strengths	Very popular, huge learning material, one language everywhere, flexible schema that is quick for prototyping, JSON data from database to browser.
Trade-offs	Relationships and multi-record consistency need extra care. Without enforced foreign keys, data like requests pointing to deleted users can appear. Inventory allocation with multi-document transactions is possible but less natural.
Learning complexity	Low to medium.
Backend flexibility	High; same Express flexibility as PERN.
Database suitability	Fair. Good for flexible documents; FloodRelief's data is relational and consistency-critical.
Performance	Both can perform well. Performance depends mostly on indexes, queries and design, not on the stack name.
Suitability	A workable choice, but PostgreSQL matches the project's relationships and inventory rules better.

2. MEAN: MongoDB + Express + Angular + Node

Technologies	MongoDB, Express, Angular, Node.js. TypeScript is standard in Angular.
Strengths	Angular is a full framework with routing, forms and dependency injection built in; good for large structured interfaces.
Trade-offs	Same MongoDB trade-offs as MERN, plus Angular's larger learning curve (modules, decorators, RxJS). Note: Angular alone does not make a project MEAN; all four parts are required.
Learning complexity	Medium to high.
Backend flexibility	High.
Database suitability	Fair.
Performance	Comparable; mostly depends on implementation.
Suitability	Reasonable if you specifically want to learn Angular, but it has both the heavier frontend and the less suitable database for this project.

3. Angular + Spring Boot + PostgreSQL (the original PDF choice)

Technologies	Angular + TypeScript, Java + Spring Boot, Spring Data JPA/Hibernate, PostgreSQL/PostGIS, Spring Security.
Strengths	Mature, highly structured, excellent for large business systems with many rules and roles, strong tooling for validation, security and testing, and strongly typed on both sides. Different from the common JavaScript portfolios.
Trade-offs	Two quite different ecosystems to learn (TypeScript and Java), more boilerplate, longer setup, heavier for a beginner.
Learning complexity	High.
Backend flexibility	High but with strong conventions.
Database suitability	Excellent (PostgreSQL).
Performance	Java is very capable for concurrent, heavy workloads; FloodRelief's scale does not require it.
Suitability	A solid choice and a valid one. It was replaced because the learning cost is higher than the project's needs. Consider it if your goal is to learn enterprise Java.

4. React + NestJS + PostgreSQL

Technologies	React, NestJS (a TypeScript backend framework built on Node, often on top of Express), PostgreSQL, typically with TypeORM, Prisma or similar.
Strengths	Enforces a modular structure (modules, controllers, providers, dependency injection), integrates validation and guards, TypeScript from the start, scales well in code organization.
Trade-offs	More concepts to learn up front (decorators, dependency injection, modules). Beginners can use the structure without understanding why it exists.
Learning complexity	Medium.
Backend flexibility	Structured, less free-form than Express, still very capable.
Database suitability	Excellent.
Performance	Similar to Express (it runs on the same runtime).
Suitability	Very good. A natural next step: learning Express first makes NestJS easier to understand.

5. Next.js + Supabase

Technologies	Next.js (React framework with server rendering and API routes), Supabase (hosted PostgreSQL with built-in auth, storage, row-level security and realtime).
Strengths	Very fast to start; real PostgreSQL underneath; auth and file storage included; less infrastructure to manage.
Trade-offs	You write less of the backend yourself, so it shows less backend API design in a portfolio. Row-level security policies have their own learning curve. Multi-step allocation logic usually needs database functions or server routes. Reliance on a hosted provider.
Learning complexity	Low to medium to start; deeper topics appear later.
Backend flexibility	Medium. Powerful but shaped by the platform.
Database suitability	Excellent (PostgreSQL).
Performance	Good for typical applications; depends on queries and hosting region.
Suitability	Good for a quick prototype. Because the goal here includes learning a clear frontend/backend separation and API development, PERN gives more direct learning.

6. Laravel + React

Technologies	PHP with the Laravel framework (routing, Eloquent ORM, migrations, validation, queues, auth) and React, with PostgreSQL as an option.
Strengths	Extremely productive, batteries included, excellent documentation, strong conventions for migrations, validation and authorization policies.
Trade-offs	Adds a second language (PHP) alongside TypeScript. Types are not shared between frontend and backend as easily. PHP hosting and tooling differ from the Node ecosystem.
Learning complexity	Medium.
Backend flexibility	High within Laravel's conventions.
Database suitability	Very good.
Performance	Perfectly adequate for this scale.
Suitability	A strong choice if you know or want to learn PHP. Not selected because TypeScript on both sides is simpler for a beginner.

6.3 Why PERN is a reasonable balance

Criterion	Why PERN fits
Beginner learning curve	One language (TypeScript) across the whole project; Express is small and easy to understand; React has abundant learning resources.
Clear frontend/backend separation	React and Express are separate applications that communicate through an API, which makes the architecture easy to see and explain.
API development experience	You design routes, middleware, validation and error handling yourself, which teaches how APIs really work.
PostgreSQL requirements	Relational data, constraints, transactions and PostGIS are exactly what inventory and location features need.
Portfolio value	Popular, recognizable stack that demonstrates full-stack skills, SQL, security and testing.
Development flexibility	Few hard conventions: you can choose your own libraries, grow the structure, or move to NestJS later.

WARNING
PERN is a reasonable balance for this project and this learner, not a universally better stack. Its main trade-off is flexibility without guard rails: Express does not enforce a structure, so you must apply the folder structure and layering from chapter 14 with discipline.

6.4 When another stack might be a better choice

If your situation is...	Consider...	Because
You want the fastest prototype with managed auth and hosting	Next.js + Supabase	Less setup; but you give up some backend learning.
You want to learn enterprise Java and strict typing	Angular or React + Spring Boot + PostgreSQL	Mature, structured ecosystem.
You want enforced structure on a large TypeScript backend	React + NestJS + PostgreSQL	Modules and dependency injection.
You or your team already know PHP	Laravel + React	Productivity from existing knowledge.
Your data is truly document-shaped and changes shape often	MERN	Flexible schema.
You need server-rendered public pages for SEO	Next.js	Built-in server rendering. FloodRelief's private workflow does not need it.

7. Database Architecture

This chapter proposes a relational model for FloodRelief in PostgreSQL. It starts from the eleven tables named in the project brief and adds a few recommended tables that make the described features (duplicates, location hierarchy, inventory ledger, proof of delivery, teams, audit) possible. It is a **starting point**: columns and names can change when you meet real implementation details.

7.1 Design principles

- **Let the database enforce rules.** Foreign keys, NOT NULL, UNIQUE and CHECK constraints protect integrity even if code has bugs.
- **Use UUID primary keys** (or bigint identity). UUIDs are not guessable, which prevents people from enumerating request numbers in URLs. A separate human-friendly reference (FR-000042) can be shown to users.
- **Use** `timestamptz` for all times and store UTC. Convert to Asia/Dhaka only in the interface.
- **Store statuses as text with CHECK constraints** (or PostgreSQL enums). CHECK lists are easier to change with migrations.
- **Never physically delete important records.** Use statuses (cancelled, rejected, suspended) so history stays complete.
- **Record who and when.** Include `created_at`, `updated_at` and the acting user where relevant.
- **Keep exact and public locations separate.** Store exact coordinates privately and an approximate version for wider display.

7.2 Entity relationship diagrams

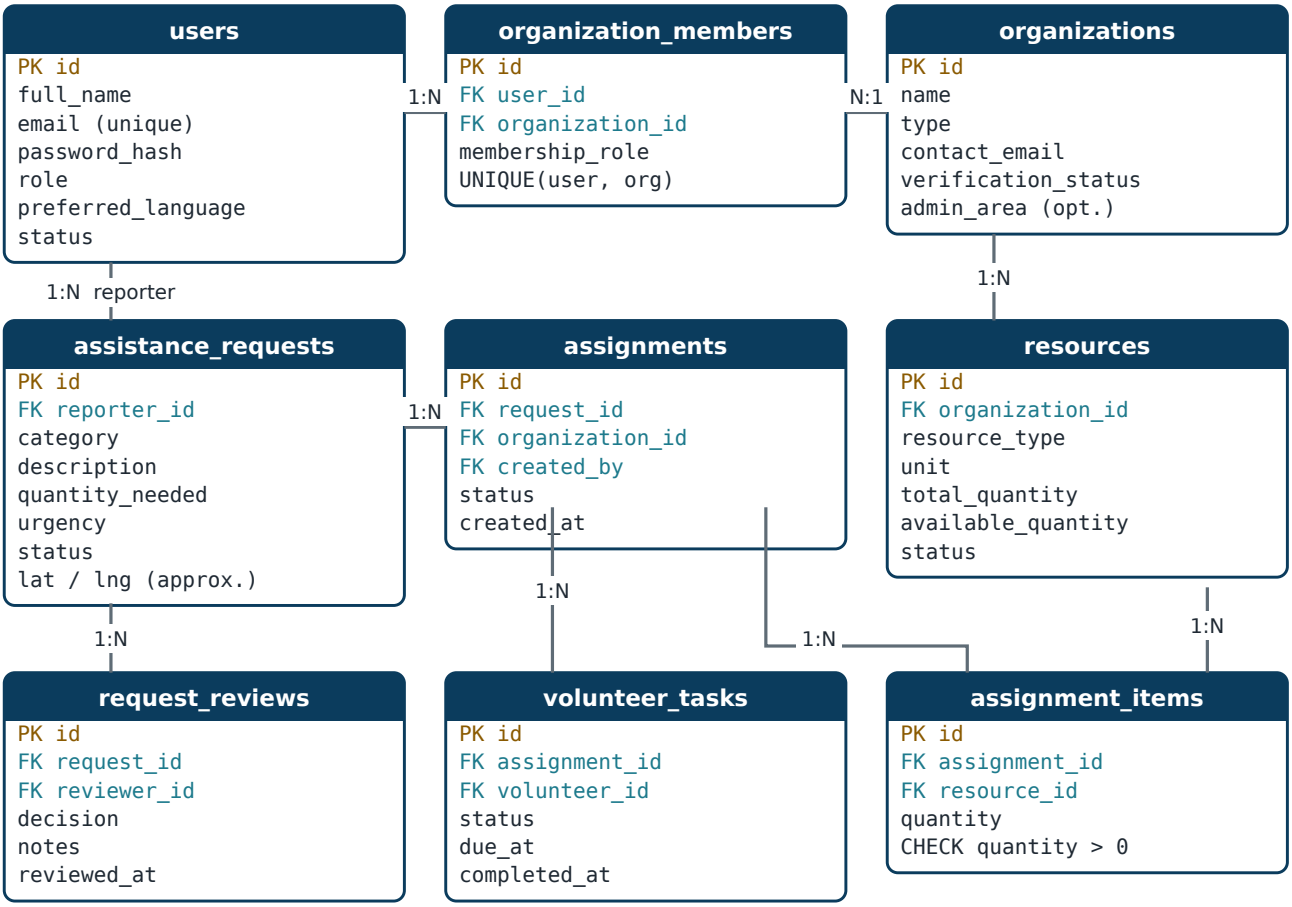
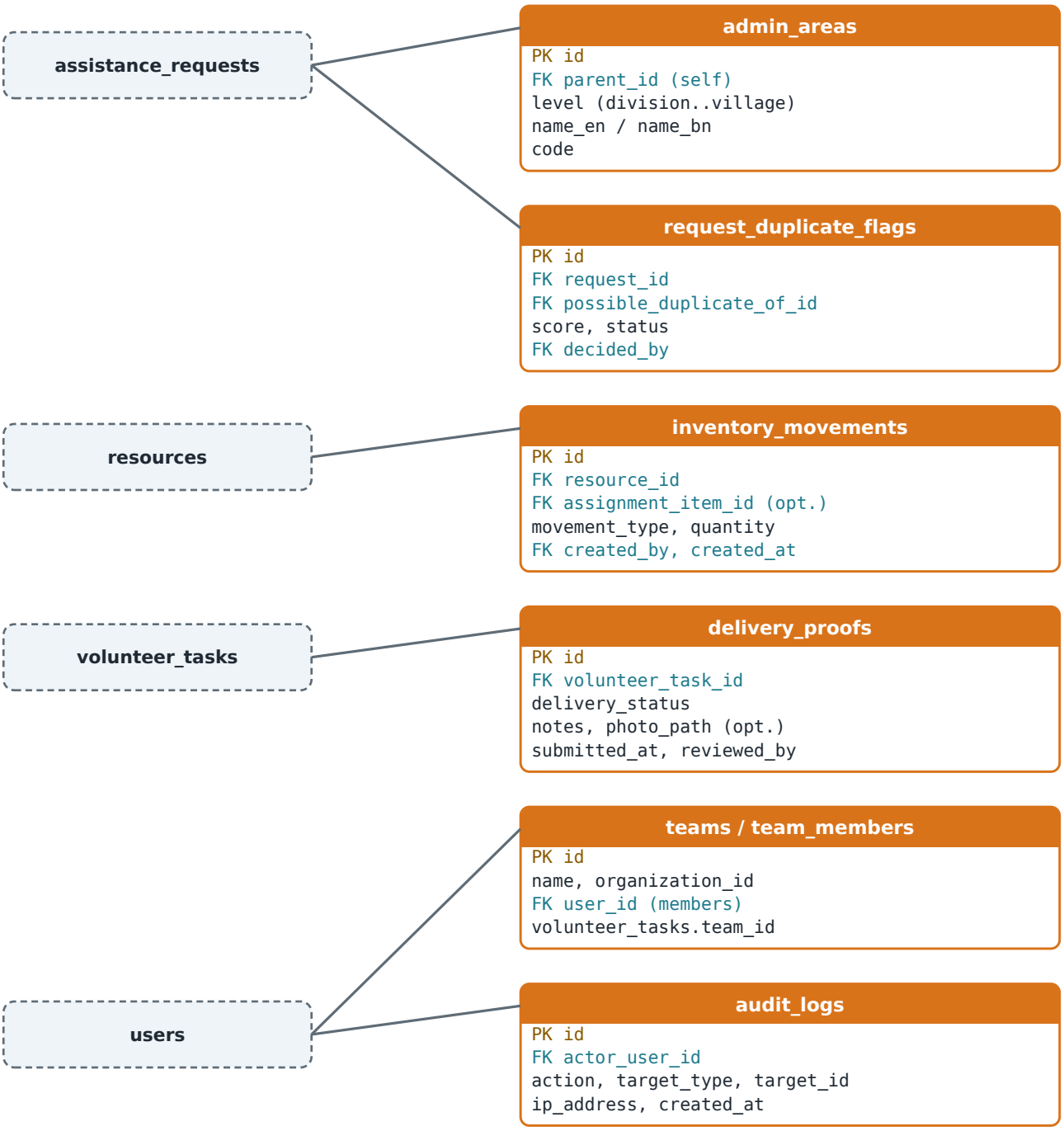


Figure 7.1: Core tables from the brief and their main relationships.



Dashed grey = existing core tables (shown for context).
Orange = recommended additions to the original table list.

Figure 7.2: Recommended additional tables (orange) and the core tables they connect to.

7.3 Status values (proposed)

Entity	Allowed statuses	Notes
`assistance_requests.status`	submitted, needs_info, verified, rejected, duplicate, assigned, in_progress, fulfilled, cancelled	Based on statuses mentioned in the original PDF; some names are recommendations.
`assistance_requests.urgency`	low, medium, high, critical	Set by the reporter; coordinators may correct it.
`assignments.status`	planned, in_progress, completed, cancelled	planned = resources allocated, tasks not finished.
`volunteer_tasks.status`	assigned, accepted, declined, in_progress, completed, unable_to_complete	From the original PDF with declined added.
`organizations.verification_status`	pending, approved, rejected, suspended	
`resources.status`	available, reserved, unavailable	Quantities decide real availability.
`users.status`	active, suspended	
`users.role`	reporter, volunteer, org_member, coordinator, admin	One role per account in the MVP.

7.4 The eleven core tables

users

Purpose	Accounts and roles for everyone who signs in. Passwords are never stored, only a hash.
Important columns	`id`, `full_name`, `email`, phone (optional, restricted), `password_hash`, `role`, `preferred_language` (en or bn), `status`, `created_at`, `updated_at`, `last_login_at`
Primary key	`id` (UUID)
Foreign keys	None.
Relationships	One user has many requests, tasks, notifications and organization memberships.
Constraints	`email` UNIQUE (case-insensitive, for example with citext or a lower() index); NOT NULL on name, email, password_hash, role; CHECK on role and status.
Design notes	Do not return `password_hash` from any query that feeds an API response.

organizations

Purpose	Relief groups that own resources and take responsibility for assignments.
Important columns	id, name, type, contact_email, contact_phone, admin_area_id, verification_status, created_at
Primary key	id
Foreign keys	admin_area_id → admin_areas.id (optional)
Relationships	One organization has many members, resources and assignments.
Constraints	NOT NULL name; CHECK on verification_status; UNIQUE (name, admin_area_id) to reduce accidental duplicates.

organization_members

Purpose	Connects users to organizations (many-to-many) and stores the membership role.
Important columns	id, user_id, organization_id, membership_role (owner, manager, member), joined_at
Primary key	id
Foreign keys	user_id → users.id; organization_id → organizations.id
Relationships	Resolves the many-to-many relationship between users and organizations.
Constraints	UNIQUE (user_id, organization_id); CHECK on membership_role; ON DELETE CASCADE only for removing a membership.

assistance_requests

Purpose	The reported needs: the central table of the project.
Important columns	id, reference (readable code), reporter_id, category, description, quantity_needed, unit, urgency, status, admin_area_id, lat/lng (exact, private), public_lat/public_lng (approximate), contact_phone (restricted), created_at, updated_at
Primary key	id
Foreign keys	reporter_id → users.id; admin_area_id → admin_areas.id; duplicate_of_id → assistance_requests.id (optional, set when a duplicate is confirmed)
Relationships	Many requests per reporter; a request has many reviews, assignments and history rows.
Constraints	NOT NULL on category, urgency, status, reporter_id; CHECK quantity_needed > 0; CHECK on status, urgency and latitude/longitude ranges; reference UNIQUE.
Design notes	Indexes: (status, urgency), (admin_area_id, status), (created_at DESC), (reporter_id). Add a spatial index only if PostGIS is adopted.

request_reviews

Purpose	Stores every verification decision. A request can be reviewed more than once (for example needs_info, then verified).
Important columns	id, request_id, reviewer_id, decision (verified, rejected, needs_info), notes, reviewed_at
Primary key	id
Foreign keys	request_id → assistance_requests.id; reviewer_id → users.id
Relationships	Many reviews per request.
Constraints	NOT NULL on decision and reviewer_id; CHECK on decision; notes required (application rule) for rejection and needs_info; no update or delete in the application.

resources

Purpose	What an organization holds: stock, capacity or equipment. The numbers used for allocation.
Important columns	id, organization_id, resource_type (drinking_water, dry_food, medicine, boat, shelter_space...), name, unit, total_quantity, available_quantity, admin_area_id, status, updated_at
Primary key	id
Foreign keys	organization_id → organizations.id; admin_area_id → admin_areas.id
Relationships	Many resources per organization; referenced by assignment_items and inventory_movements.
Constraints	CHECK (total_quantity >= 0); CHECK (available_quantity >= 0); CHECK (available_quantity <= total_quantity).
Design notes	Total = what the organization physically has. Available = total minus what is currently allocated. The CHECK constraints are the safety net against over-allocation.

assignments

Purpose	A relief job created for one verified request. It groups allocated resources and volunteer tasks.
Important columns	id, request_id, organization_id (responsible organization, optional), created_by, status, notes, created_at, updated_at
Primary key	id
Foreign keys	request_id → assistance_requests.id; organization_id → organizations.id; created_by → users.id
Relationships	One request can have more than one assignment (for example a follow-up delivery). One assignment has many items and many tasks.
Constraints	NOT NULL on request_id and created_by; CHECK on status.
Design notes	The request must be in the verified (or assigned) status when an assignment is created; the service enforces this.

assignment_items

Purpose	Which resource, and how much of it, is allocated to an assignment. This is what makes "60 from A and 40 from B" a recorded fact.
Important columns	id, assignment_id, resource_id, quantity, status (allocated, delivered, released), created_at
Primary key	id
Foreign keys	assignment_id → assignments.id (ON DELETE CASCADE only if assignments are ever hard-deleted, which should not happen); resource_id → resources.id
Relationships	Resolves the many-to-many relationship between assignments and resources.
Constraints	CHECK (quantity > 0); UNIQUE (assignment_id, resource_id) so one resource appears once per assignment.

volunteer_tasks

Purpose	Work given to a volunteer (or team) as part of an assignment.
Important columns	id, assignment_id, volunteer_id, team_id (optional), status, instructions, due_at, accepted_at, completed_at, created_by
Primary key	id
Foreign keys	assignment_id → assignments.id; volunteer_id → users.id; team_id → teams.id (optional)
Relationships	Many tasks per assignment; one volunteer can have many tasks.
Constraints	CHECK on status; CHECK that either volunteer_id or team_id is set; index on (volunteer_id, status).

status_history

Purpose	An insert-only log of status changes for requests, assignments and tasks, used for timelines and accountability.
Important columns	id, entity_type (request, assignment, task, organization), entity_id, old_status, new_status, changed_by, reason, changed_at
Primary key	id
Foreign keys	changed_by → users.id. entity_id is polymorphic and has no database foreign key.
Relationships	Logically belongs to requests, assignments and tasks.
Constraints	NOT NULL on entity_type, entity_id, new_status, changed_at; index on (entity_type, entity_id, changed_at).
Design notes	Trade-off: a single polymorphic table is simple and gives one timeline query, but the database cannot enforce the link. The service must always write history in the same transaction. Alternative: one history table per entity.

notifications

Purpose	In-app messages for users.
Important columns	id, user_id, title, message, link_type/link_id (what it refers to), read_at, created_at
Primary key	id
Foreign keys	user_id → users.id
Relationships	Many notifications per user.
Constraints	NOT NULL on user_id, title, message; index on (user_id, read_at, created_at DESC) for the unread badge.

7.5 Recommended additional tables

> **RECOMMENDATION (not in the original document)**
> These tables are not in the original table list. They are suggested because features described in the brief (duplicates, location hierarchy, inventory accountability, proof of delivery, team management, audit logs) need somewhere to be stored. Add them only when you reach the phase that needs them.

admin_areas

Purpose	The Bangladesh location hierarchy (division, district, upazila, union, village) in one self-referencing table.
Important columns	id, parent_id, level, name_en, name_bn, code, optional centre point.
Primary key	id
Foreign keys	parent_id → admin_areas.id
Relationships	Referenced by requests, organizations and resources.
Constraints	CHECK on level; UNIQUE (parent_id, name_en); index on parent_id.

request_duplicate_flags

Purpose	Duplicate suggestions and decisions.
Important columns	id, request_id, possible_duplicate_of_id, score, reasons, status (suggested, confirmed, dismissed), decided_by, decided_at
Primary key	id
Foreign keys	Two FKs to assistance_requests.id; decided_by → users.id
Relationships	Many flags per request.
Constraints	CHECK (request_id <> possible_duplicate_of_id); UNIQUE (request_id, possible_duplicate_of_id).

inventory_movements

Purpose	The inventory ledger: each change to a resource quantity as a row.
Important columns	id, resource_id, assignment_item_id (optional), movement_type, quantity_delta, reason, created_by, created_at
Primary key	id
Foreign keys	resource_id → resources.id; assignment_item_id → assignment_items.id; created_by → users.id
Relationships	Many movements per resource.
Constraints	Insert-only; CHECK on movement_type; index on (resource_id, created_at).

delivery_proofs

Purpose	Proof of delivery submitted by volunteers and reviewed by coordinators.
Important columns	id, volunteer_task_id, delivery_status (delivered, partial, failed), delivered_at, notes, photo_path (optional), review_status, reviewed_by, reviewed_at
Primary key	id
Foreign keys	volunteer_task_id → volunteer_tasks.id; reviewed_by → users.id
Relationships	Usually one proof per task, but several are allowed for partial deliveries.
Constraints	CHECK on delivery_status and review_status. Store only a storage path, never the image bytes in the table.

teams / team_members

Purpose	Groups of volunteers.
Important columns	teams: id, name, organization_id. team_members: team_id, user_id, role.
Primary key	teams.id; team_members uses (team_id, user_id) as a composite key.
Foreign keys	teams.organization_id → organizations; team_members FKs to teams and users.
Relationships	Many-to-many between users and teams.
Constraints	UNIQUE team name per organization.

audit_logs

Purpose	Security and administration events that are not status changes (login, failed login, role change, export).
Important columns	id, actor_user_id, action, target_type, target_id, ip_address, details (JSON, no secrets), created_at
Primary key	id
Foreign keys	actor_user_id → users.id (nullable for failed logins)
Relationships	Many entries per user.
Constraints	Insert-only; index on (created_at) and (actor_user_id).

7.6 Foreign key summary

Child table.column	References	Meaning
organization_members.user_id	users.id	Who is the member
organization_members.organization_id	organizations.id	Which organization
assistance_requests.reporter_id	users.id	Who reported
assistance_requests.admin_area_id	admin_areas.id	Where
request_reviews.request_id / reviewer_id	assistance_requests.id / users.id	What was reviewed, by whom
resources.organization_id	organizations.id	Owner of the stock
assignments.request_id	assistance_requests.id	Which need it serves
assignment_items.assignment_id / resource_id	assignments.id / resources.id	What is allocated and from where
volunteer_tasks.assignment_id / volunteer_id	assignments.id / users.id	Task and its volunteer
status_history.changed_by	users.id	Who changed it
notifications.user_id	users.id	Recipient

7.7 Example: the water allocation, step by step

Starting data: a verified request (id R1) needs **100 bottles** of drinking water. Organization A has resource **RA** with 60 available; Organization B has resource **RB** with 50 available.

What the coordinator confirms: 60 from RA and 40 from RB. **What the database ends up with:**

Table	After the transaction
assignments	One new row for request R1, status planned.
assignment_items	Two rows: (RA, 60) and (RB, 40).
resources	RA: available 0 (total still 60). RB: available 10 (total still 50).
inventory_movements	Two rows with type "allocation": RA −60, RB −40.
assistance_requests	Status changed from verified to assigned.
status_history	One row: verified → assigned, changed_by the coordinator.
notifications	Rows for the reporter and for the organization members of A and B.

7.8 How transactions prevent over-allocation

The problem (a race condition). Two coordinators work at the same time. Both read that RA has 60 available. Coordinator 1 allocates 60. Coordinator 2 allocates 40 a moment later, based on the old number they read. Without protection the database would now say 100 bottles are promised from stock that holds only 60.

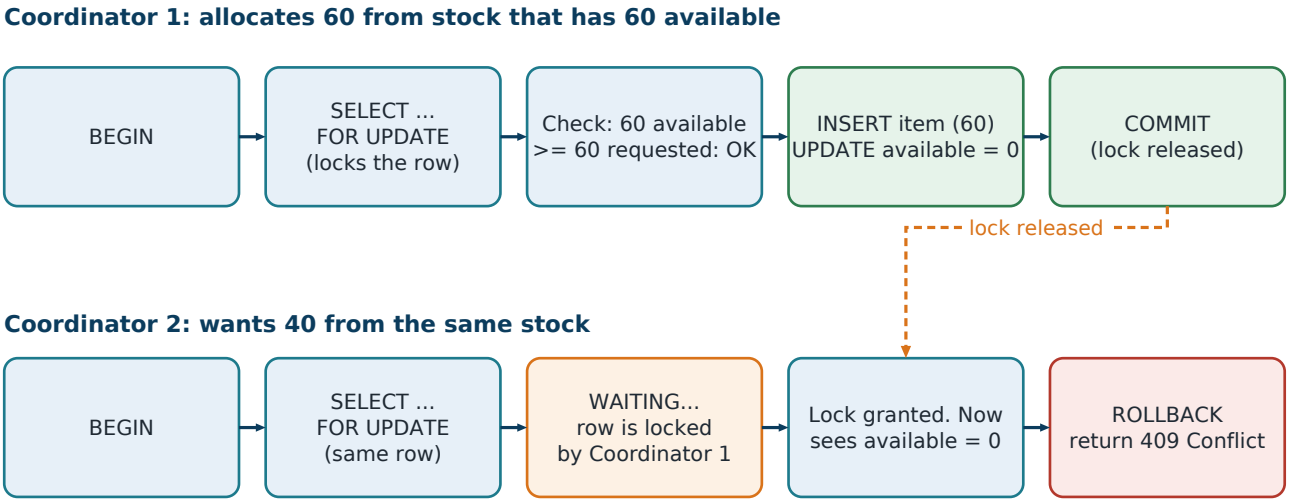


Figure 7.3: Two coordinators competing for the same stock. Row locking makes them go one after the other.

The solution has three layers:

1. **A transaction** wraps the whole allocation so it is all-or-nothing.
2. **Row locking.** `SELECT ... FOR UPDATE` locks the resource rows until the transaction ends. A second transaction that wants the same rows waits, then sees the updated quantity.
3. **A CHECK constraint** (`available_quantity >= 0`) makes the database refuse any final state with negative stock, even if the application code is wrong.

Resources should always be locked in a consistent order (for example sorted by id) so two transactions never wait for each other in a circle (a **deadlock**).

```
BEGIN;

-- 1. Lock the resource rows in a fixed order
SELECT id, available_quantity FROM resources
WHERE id = ANY($1::uuid[]) ORDER BY id FOR UPDATE;

-- 2. Application check: request is 'verified' and every quantity <= available

-- 3. Record the allocation
INSERT INTO assignments (request_id, created_by, status)
VALUES ($2, $3, 'planned') RETURNING id;

INSERT INTO assignment_items (assignment_id, resource_id, quantity)
VALUES ($4, $5, 60), ($4, $6, 40);

-- 4. Reduce stock (the WHERE clause is a second guard)
UPDATE resources SET available_quantity = available_quantity - 60, updated_at = now()
WHERE id = $5 AND available_quantity >= 60;      -- 0 rows updated => abort

INSERT INTO inventory_movements (resource_id, movement_type, quantity_delta, created_by)
VALUES ($5, 'allocation', -60, $3);

-- 5. Move the request forward and write the history
UPDATE assistance_requests SET status = 'assigned' WHERE id = $2 AND status = 'verified';
INSERT INTO status_history (entity_type, entity_id, old_status, new_status, changed_by)
VALUES ('request', $2, 'verified', 'assigned', $3);

COMMIT;   -- on any error: ROLLBACK
```

In Node, one **client** must be used for every statement of the transaction (not `pool.query`, which may use a different connection each time):

```
export async function createAssignment(user: AuthUser, input: NewAssignment) {
  const client = await pool.connect();
  try {
    await client.query("BEGIN");
    // ... the statements above, all with client.query(...)
    await client.query("COMMIT");
    return result;
  } catch (err) {
    await client.query("ROLLBACK");
    throw err;                 // the error handler turns it into 409 or 500
  } finally {
    client.release();          // always give the connection back to the pool
  }
}
```

> **NOTE**
> PostgreSQL's default isolation level (READ COMMITTED) together with `FOR UPDATE` row locks is enough for this use case. You do not need to learn the stricter isolation levels to build the MVP.

7.9 Migrations

A **migration** is a versioned script that changes the database structure (create a table, add a column). Migrations are stored in Git, run in order, and are the same in development and production. Never edit the production database structure by hand. Suggested tool: **node-pg-migrate** (or Prisma Migrate, Knex or Drizzle Kit if you choose those). Write a seed script that loads the area hierarchy and **synthetic** demo data.

8. API Architecture

The API is a **REST-style JSON API** served by Express under the `/api` prefix. This chapter lists the proposed endpoints grouped by area. The original PDF provided example endpoint groups; the complete list below is an expanded recommendation and should be finalized during implementation.

8.1 Conventions

- **Resources are nouns** (`/api/requests`), actions are HTTP methods. Actions that are not simple updates use a sub-path (`/api/assignments/:id/cancel`).
- **JSON in, JSON out.** Dates are ISO 8601 strings in UTC.
- **Authentication by cookie.** "Auth" below means a valid session. State-changing requests also carry a CSRF token (chapter 9).
- **Pagination:** `?page=1&pageSize=20` returns `{ items, page, pageSize, total }`.
- **Filtering:** query parameters such as `?status=verified&urgency=critical&areaId=...`.
- **Errors** share one shape: `{ "error": { "code": "STOCK_INSUFFICIENT", "message": "...", "details": [...] } }`.
- **Permissions** in the tables name the minimum roles; "own" means ownership is checked in addition to the role.

8.2 Authentication

Method	URL	Purpose	Permission
POST	/api/auth/register	Create a reporter or volunteer account.	Public (rate limited)
POST	/api/auth/login	Sign in; sets the session cookie.	Public (rate limited)
POST	/api/auth/logout	Clear the session cookie.	Auth
GET	/api/auth/me	Return the current user and role.	Auth
GET	/api/auth/csrf	Return a CSRF token for forms.	Auth or public

8.3 Users

Method	URL	Purpose	Permission
GET	/api/users/me	View own profile.	Auth
PATCH	/api/users/me	Update own name, phone, language.	Auth (own)
GET	/api/users	List users (filter by role).	Coordinator (volunteers only), Admin
GET	/api/users/:id	View a user.	Admin
PATCH	/api/users/:id/role	Change a user's role.	Admin
PATCH	/api/users/:id/status	Suspend or reactivate a user.	Admin

8.4 Organizations

Method	URL	Purpose	Permission
POST	/api/organizations	Register an organization (pending).	Auth
GET	/api/organizations	List organizations.	Coordinator, Admin; members see own
GET	/api/organizations/:id	View an organization.	Member, Coordinator, Admin
PATCH	/api/organizations/:id	Edit organization profile.	Owner or manager, Admin
PATCH	/api/organizations/:id/verification	Approve, reject or suspend.	Admin
GET	/api/organizations/:id/members	List members.	Owner or manager, Admin
POST	/api/organizations/:id/members	Add a member.	Owner or manager
DELETE	/api/organizations/:id/members/:userId	Remove a member.	Owner or manager

8.5 Assistance requests

Method	URL	Purpose	Permission
POST	/api/requests	Submit a new request.	Reporter, Coordinator, Admin
GET	/api/requests	List with filters and pagination.	Reporter (own), Coordinator (area), Admin
GET	/api/requests/:id	Request details.	Own, Coordinator, Admin; Volunteer or Org if linked
PATCH	/api/requests/:id	Edit while still submitted or needs_info.	Reporter (own), Coordinator
POST	/api/requests/:id/cancel	Cancel a request.	Reporter (own), Coordinator
GET	/api/requests/:id/history	Status timeline.	Own, Coordinator, Admin
GET	/api/requests/map	Markers inside a bounding box, with filters.	Coordinator, Admin
GET	/api/areas	Administrative areas by parent (cascading drop-downs).	Auth

8.6 Verification and duplicates

Method	URL	Purpose	Permission
PATCH	/api/requests/:id/verification	Verify, reject or ask for more information.	Coordinator, Admin
GET	/api/requests/:id/duplicates	Duplicate suggestions for a request.	Coordinator, Admin
POST	/api/requests/:id/duplicate-flags	Flag a possible duplicate manually.	Coordinator, Admin
PATCH	/api/duplicate-flags/:id	Confirm or dismiss a suggestion.	Coordinator, Admin

8.7 Resources

Method	URL	Purpose	Permission
POST	/api/resources	Create a resource record.	Org member (own org)
GET	/api/resources	List resources with filters (type, area, organization).	Org member (own), Coordinator, Admin
GET	/api/resources/:id	Resource details.	Org member (own), Coordinator, Admin
PATCH	/api/resources/:id	Edit descriptive fields.	Org member (own org)
POST	/api/resources/:id/adjustments	Adjust quantity with a reason (writes ledger).	Org member (own org)
GET	/api/resources/:id/movements	Inventory ledger for the resource.	Org member (own), Coordinator, Admin

8.8 Assignments

Method	URL	Purpose	Permission
POST	/api/assignments	Allocate resources and create an assignment (transaction).	Coordinator, Admin
GET	/api/assignments	List assignments.	Coordinator, Admin, Org member (own)
GET	/api/assignments/:id	Assignment details with items and tasks.	Coordinator, Admin, linked Org or Volunteer
PATCH	/api/assignments/:id/status	Move the assignment status forward.	Coordinator, Admin
POST	/api/assignments/:id/cancel	Cancel and release the allocated stock.	Coordinator, Admin

8.9 Volunteer tasks, teams and proof of delivery

Method	URL	Purpose	Permission
POST	/api/assignments/:id/tasks	Create a task for a volunteer or team.	Coordinator, Admin, Org manager
GET	/api/volunteer/tasks	List my tasks.	Volunteer (own)
GET	/api/volunteer/tasks/:id	Task details.	Volunteer (own), Coordinator
PATCH	/api/volunteer/tasks/:id	Accept, decline, start, complete.	Volunteer (own)
POST	/api/volunteer/tasks/:id/proof	Submit proof of delivery (multipart for photo).	Volunteer (own)
GET	/api/tasks/:id/proof	View proof.	Coordinator, Admin
PATCH	/api/tasks/:id/proof/review	Accept or reject proof.	Coordinator, Admin
POST	/api/teams	Create a team.	Org manager
POST	/api/teams/:id/members	Add a volunteer to a team.	Org manager

8.10 Notifications

Method	URL	Purpose	Permission
GET	/api/notifications	List my notifications.	Auth (own)
GET	/api/notifications/unread-count	Unread number for the bell icon.	Auth (own)
PATCH	/api/notifications/:id/read	Mark one as read.	Auth (own)
POST	/api/notifications/read-all	Mark all as read.	Auth (own)

8.11 Dashboard and administration

Method	URL	Purpose	Permission
GET	/api/dashboard/summary	Counts by status, pending allocation, completed.	Coordinator, Admin, Org (own summary)
GET	/api/dashboard/by-area	Counts grouped by administrative area.	Coordinator, Admin
GET	/api/dashboard/resources	Available stock by resource type.	Coordinator, Admin
GET	/api/admin/audit-logs	Search audit logs.	Admin

8.12 Example requests and responses

Login

```
POST /api/auth/login
{ "email": "reporter@example.test", "password": "********" }

200 OK   (Set-Cookie: session=...; HttpOnly; Secure; SameSite=Lax)
{ "user": { "id": "u_01", "fullName": "Demo Reporter", "role": "reporter" } }

401 Unauthorized
{ "error": { "code": "INVALID_CREDENTIALS", "message": "Email or password is incorrect" } }
```

Verify a request

```
PATCH /api/requests/7c2e.../verification
{ "decision": "verified", "notes": "Confirmed by phone with the local contact" }

200 OK
{ "id": "7c2e...", "status": "verified", "reviewedBy": "u_17",
  "reviewedAt": "2026-11-08T08:20:00Z" }

403 Forbidden  (caller is a volunteer)    409 Conflict  (request is already verified)
```

Allocate resources

```
POST /api/assignments
{ "requestId": "7c2e...",
  "items": [ { "resourceId": "ra11...", "quantity": 60 },
             { "resourceId": "rb22...", "quantity": 40 } ] }

201 Created
{ "id": "as01...", "status": "planned",
  "items": [ { "resourceId": "ra11...", "quantity": 60 },
             { "resourceId": "rb22...", "quantity": 40 } ] }

409 Conflict
{ "error": { "code": "STOCK_INSUFFICIENT",
             "message": "Only 10 available for resource rb22...",
             "details": [ { "resourceId": "rb22...", "available": 10 } ] } }
```

Update a task

```
PATCH /api/volunteer/tasks/9
{ "status": "completed" }

200 OK  { "id": 9, "status": "completed", "completedAt": "2026-11-08T11:05:00Z" }
422 Unprocessable Content
{ "error": { "code": "INVALID_TRANSITION",
             "message": "assigned -> completed is not allowed" } }
```

Map markers

```
GET /api/requests/map?bbox=91.80,24.85,91.95,24.95&status=verified&urgency=critical,high

200 OK
{ "items": [ { "id": "7c2e...", "lat": 24.8801, "lng": 91.8702,
               "urgency": "critical", "category": "water", "status": "verified" } ],
  "truncated": false }
```

8.13 Common HTTP status codes in this API

The full list was introduced in chapter 5. In this API, the most important distinctions are:

Situation	Status	Reason
Missing or expired session	401	The server does not know who you are.
Wrong role, or someone else's record	403 (or 404)	The server knows you but refuses. Use 404 when revealing that the record exists would itself leak information.
Invalid field values	422 (or 400 for malformed JSON)	Zod validation failed; include field-level details.
Not enough stock; invalid status transition; already verified	409	The request is valid but conflicts with the current state.
Too many attempts	429	Rate limit; include a Retry-After header.
Unexpected exception	500	A bug; return a generic message and log the details on the server.

9. Security Architecture

Disaster data is sensitive: it contains locations, phone numbers and the names of people in need. Security is therefore a design concern from the first day, not a final polish step. This chapter explains each security measure in simple language, why it is needed, and where it lives in the architecture.

> **WARNING**
> **Three non-negotiable rules.** (1) Never store plaintext passwords. (2) Never put database credentials or secrets in frontend code or in Git. (3) Frontend validation and hidden buttons are **not** security: the backend must validate and authorize every request, every time.

9.1 Where each measure lives

Layer / place	Security measures
Browser (React)	Escape output (React does this by default), never store tokens in localStorage, friendly validation (usability only).
HTTPS transport	Encrypts traffic between browser and server; required for Secure cookies.
Express middleware chain	Helmet, CORS, rate limiting, body size limits, cookie parsing, authentication, CSRF check, role check, Zod validation.
Services	Ownership and area checks, business-rule checks, audit logging.
Repositories	Parameterized SQL only.
PostgreSQL	Constraints, a least-privilege database user, backups, no public network exposure.
Environment	Secrets in environment variables; separate dev and production values.

A typical order of middleware in `app.ts`:

```
app.use(helmet());                       // security headers
app.use(cors({ origin: env.FRONTEND_ORIGIN, credentials: true }));
app.use(express.json({ limit: "100kb" }));
app.use(cookieParser());
app.use("/api", generalRateLimiter);
app.use("/api/auth", authRateLimiter);   // stricter
app.use(attachUserFromSession);          // sets req.user when a valid session exists
app.use("/api", csrfProtection);         // for state-changing methods
app.use("/api", routes);                 // routes add requireAuth, requireRole
app.use(errorHandler);                   // never leak stack traces
```

9.2 Password hashing: Argon2id or bcrypt

A **hash** is a one-way scrambling of a password. The database stores the hash; at login the server hashes the typed password and compares. If the database leaks, attackers do not directly get passwords. Plain hashes such as SHA-256 are too fast; password hashing must be deliberately **slow and salted**. **Argon2id** is the modern recommendation; **bcrypt** is older but still acceptable. Use a well-maintained library (`argon2` or `bcrypt`), keep default or recommended cost settings, and never write your own scheme.

```
import argon2 from "argon2";
```

```
const hash = await argon2.hash(plainPassword, { type: argon2.argon2id });
// store `hash` in users.password_hash
const ok = await argon2.verify(user.password_hash, typedPassword);
```

Also: enforce a sensible minimum length (for example 10 or more characters), never log passwords, and return the same error for "unknown email" and "wrong password".

9.3 Authentication and secure HTTP-only cookies

Authentication proves who the user is. After a successful login the server creates a session identifier (or signed token) and sends it in a **cookie**. Three cookie flags matter:

Flag	What it does	Why
`HttpOnly`	JavaScript in the page cannot read the cookie.	If an attacker injects a script (XSS), they cannot steal the session cookie.
`Secure`	Cookie is sent only over HTTPS.	Prevents sending the session over an unencrypted connection.
`SameSite=Lax (or Strict)`	Cookie is not sent on most cross-site requests.	First defence against CSRF (section 9.9).

Session or JWT? The original PDF notes that both are valid and the choice should be deliberate.

Option	How it works	Trade-offs
Server-side session (opaque id in cookie, session row in PostgreSQL)	The cookie holds a random id; the server looks it up on every request.	Easy to revoke (logout, suspend a user takes effect immediately). Needs a sessions table and one lookup per request.
Signed JWT in an HttpOnly cookie	The cookie holds a signed token with user id and expiry; the server verifies the signature.	No lookup needed, but revoking before expiry is harder. Keep tokens short-lived and rotate secrets carefully.

RECOMMENDATION (not in the original document)
Recommended for this project: **server-side sessions stored in PostgreSQL** with a random session id in an HttpOnly cookie. It is simple to understand, makes logout and account suspension reliable, and avoids storing tokens in browser storage. This adds a small `sessions` table (id hash, user_id, expires_at). If you choose JWT instead, still keep it in an HttpOnly cookie, not in localStorage.

```
res.cookie("session", sessionId, {
  httpOnly: true,
  secure: env.NODE_ENV === "production",
  sameSite: "lax",
  maxAge: 1000 * 60 * 60 * 8,      // 8 hours
  path: "/",
});
```

9.4 Authorization and RBAC

Authorization decides what an authenticated user may do. **RBAC** (role-based access control) grants permissions by role. In FloodRelief it has three steps, all on the server: (1) `requireAuth` rejects anonymous callers with 401; (2) `requireRole` rejects wrong roles with 403; (3) the service checks **ownership or area**, for example that the reporter owns the request, the volunteer owns the task, or the organization member belongs to the resource's organization.

```
// service layer: ownership check
export async function getRequestForUser(user: AuthUser, id: string) {
  const r = await requestsRepo.findById(id);
  if (!r) throw new NotFoundError();
  const allowed =
    user.role === "admin" ||
    (user.role === "coordinator" && (await isInUserArea(user, r.areaId))) ||
    (user.role === "reporter" && r.reporterId === user.id);
  if (!allowed) throw new NotFoundError();   // 404 hides that the record exists
  return toDtoFor(user, r);                  // removes fields the role must not see
}
```

9.5 Input validation with Zod

Zod is a TypeScript library for describing the exact shape of allowed data and checking incoming values at run time. TypeScript types disappear after compilation, so they cannot protect the server from a malicious request; Zod can. Validate the request body, query string and URL parameters in a `validate` middleware, and reject unknown or invalid fields.

```
export const createRequestSchema = z.object({
  category: z.enum(["water", "food", "medicine", "shelter", "transport",
                    "rescue", "other"]),
  urgency: z.enum(["low", "medium", "high", "critical"]),
  description: z.string().trim().min(10).max(1000),
  quantityNeeded: z.number().int().positive().max(100000),
  unit: z.string().trim().max(30),
  areaId: z.string().uuid(),
  lat: z.number().min(-90).max(90).optional(),
  lng: z.number().min(-180).max(180).optional(),
}).strict();       // unknown fields (for example "role" or "status") are rejected
```

> **NOTE**
> The frontend can reuse the same schema with React Hook Form for friendly instant messages. That is a convenience; the backend still validates because attackers do not use your frontend.

9.6 Helmet, rate limiting and CORS

Measure	What it is	Why FloodRelief needs it
Helmet	Express middleware that sets protective HTTP response headers (Content-Security-Policy, X-Content-Type-Options, Strict-Transport-Security and others).	Reduces common browser-based attacks such as clickjacking and content sniffing with one line of code.
Rate limiting	Limits how many requests a client may send in a time window (`express-rate-limit`).	Slows password guessing and spam requests. Use stricter limits on login, register and request submission.
CORS	A browser rule that decides which websites may call your API. Configured on the backend.	Allow only your own frontend origin, with credentials enabled. Never use "*" together with cookies.

```
import rateLimit from "express-rate-limit";
export const authRateLimiter = rateLimit({ windowMs: 15 * 60 * 1000, limit: 10 });
```

9.7 Parameterized SQL queries

SQL injection happens when user input is glued into a SQL string, letting an attacker change the meaning of the query. The defence is **parameterized queries**: the SQL text and the values are sent separately, so values can never be interpreted as SQL.

```
// DANGEROUS: never build SQL with string concatenation
pool.query("SELECT * FROM users WHERE email = '" + email + "'");

// SAFE: placeholders $1, $2 and a separate array of values
pool.query("SELECT id, role FROM users WHERE email = $1", [email]);
```

Placeholders cannot replace table or column names. When sorting by a user-chosen column, map the input to a fixed list of allowed column names in code.

9.8 Environment variables and secret management

Secrets such as the database URL, session secret and storage keys belong in **environment variables** on the server, never in source code, never in Git, and never in the React bundle. Anything bundled into the frontend is public. Commit a `.env.example` with placeholder values and add `.env` to `.gitignore`. Validate the variables at startup with Zod so the app fails fast if one is missing. In production, use your host's secret store.

```
# .env.example (committed, no real secrets)
NODE_ENV=development
PORT=4000
DATABASE_URL=postgres://user:password@localhost:5432/floodrelief
SESSION_SECRET=change-me
FRONTEND_ORIGIN=http://localhost:5173
```

> **WARNING**
> React environment variables (for example those starting with VITE_) are embedded in the public JavaScript. Put only non-secret values such as the API base URL there. If a secret is ever committed to Git, treat it as leaked: rotate it.

9.9 CSRF protection

Cross-Site Request Forgery tricks a signed-in user's browser into sending an unwanted request to your API from another website, because browsers attach cookies automatically. CSRF matters whenever authentication uses cookies, as recommended here. Layered defences:

- **SameSite=Lax or Strict cookies** (blocks most cross-site sends).
- **Host the frontend and API on the same site** (for example `app.example.org` and `api.example.org`) so SameSite cookies work without weakening them.
- **Check the Origin header** on state-changing requests (POST, PATCH, PUT, DELETE).
- **CSRF token** (double-submit or synchronizer) for state-changing requests, sent in a custom header. The older `csurf` package is deprecated; use a maintained alternative or a small custom implementation.
- **Never change state with GET.**

9.10 Audit logs

An **audit log** records who did what and when for sensitive actions: login and failed login, role changes, verification decisions, allocations, cancellations, proof reviews and exports. Status changes go to `status_history` and security events to `audit_logs`. Logs are insert-only through the application, contain no passwords or tokens, and are readable only by administrators. They support accountability, investigations and trust.

9.11 Secure file uploads

Evidence photos are the riskiest input because a file can be disguised or carry hidden data. Apply all of these:

- **Authenticate first**; only the volunteer who owns the task may upload.
- **Limit size and count** (for example 5 MB, one to three files).
- **Allow only images** (JPEG, PNG, WebP). Check the real file signature (magic bytes), not just the filename extension or the browser-provided type.
- **Rename files** to random identifiers; never use the original name in a path.
- **Strip metadata (EXIF)**, which can contain GPS coordinates, and resize with an image library such as `sharp`.
- **Store outside the public web root** (private object storage or a private folder); save only the path in `delivery_proofs.photo_path`.
- **Serve through an authorized endpoint** (`GET /api/tasks/:id/proof`) so only coordinators and administrators can view.
- **Never execute or render uploaded files** as code or HTML. Optionally scan for malware when the host provides it.
- Use libraries such as `multer` with memory storage and strict limits, then validate and process before saving.

9.12 Security checklist

Item	Done when
Passwords	Argon2id or bcrypt; no plaintext anywhere, including logs.
Cookies	HttpOnly, Secure (production), SameSite set; no tokens in localStorage.
Authorization	Every protected route has requireAuth; role and ownership checks tested.
Validation	Zod schema on every body, query and params; strict objects.
SQL	Only parameterized queries; allow-list for dynamic ORDER BY.
Headers and CORS	Helmet enabled; CORS limited to the real frontend origin.
Rate limiting	Applied globally and stricter on authentication.
Secrets	Environment variables only; .env ignored; .env.example committed.
CSRF	SameSite plus Origin check or token on state-changing requests.
Uploads	Size, type, metadata stripping, private storage, authorized access.
Logs	Audit logs for sensitive actions; error handler does not leak stack traces.
Database	Least-privilege database user; not exposed publicly; backups tested.

10. Performance and User Experience

FloodRelief should feel fast and smooth, including on phones and slow connections. Speed comes from many small, deliberate decisions in the implementation. **The stack alone does not guarantee performance**: a PERN application with unindexed queries, no pagination and thousands of simultaneously rendered markers will be slow, while a carefully built application on almost any stack will be fast. Measure first, then optimize what is actually slow.

> **RECOMMENDATION (not in the original document)**
> Do not add Redis, queues or other infrastructure for performance until a measured problem demands it. PostgreSQL with good indexes and pagination is more than enough for the scale of this project.

10.1 Frontend

Efficient React components and state management

- Keep state **as local as possible**. Lift it up only when two components need it.
- Use **TanStack Query** for server data (caching, deduplication, background refetching) instead of copying API data into your own global store.
- Use a small global store only for client-only data (for example the signed-in user or UI language); React Context is enough at first.
- Avoid unnecessary re-renders: stable keys in lists, `React.memo` for expensive list items, `useMemo` and `useCallback` only where profiling shows a benefit.
- **Debounce** search boxes (wait 300 ms after typing) so you do not send a request per keystroke.
- For very long lists, use **virtualization** (render only visible rows) with a library such as TanStack Virtual, but only if pagination is not sufficient.

Lazy loading and bundle size

- Split code by route with `React.lazy` and `Suspense` so the login page does not download the map or dashboard code.
- Load the map library only on the map page.
- Use Vite for fast builds, and inspect the bundle size periodically.
- Compress images and serve responsive sizes; use modern formats where possible.

Responsive design

Volunteers and reporters will mostly use phones. Design **mobile-first**: one-column layouts that expand on larger screens, tap targets at least about 44 px, readable font sizes, forms with the right input types (for example `inputmode="numeric"` for quantity). Test on a narrow viewport and on a throttled network. The original PDF recommends making the web app responsive before considering a separate mobile app.

Loading and error states

- Show a **skeleton or spinner** while data loads, and disable submit buttons while saving to prevent double submissions.

- Show **clear, human error messages** ("We could not save your request. Check your connection and try again") with a retry action.
- Show **empty states** that explain what to do ("No requests match these filters").
- Use **optimistic updates** sparingly: for critical actions such as allocation, wait for the server's answer.
- Handle expired sessions gracefully: redirect to login and return the user to where they were.

Accessible forms and navigation

- Every input has a visible **label** (not only a placeholder) linked with `htmlFor`.
- Error messages are attached to the field and announced to screen readers (`aria-describedby`, `aria-live`).
- Everything works with the **keyboard**: logical tab order, visible focus, dialogs that trap and restore focus.
- Use **semantic HTML** (button, nav, main, table) before reaching for ARIA attributes.
- Do not rely on **colour alone** for urgency; add text or icons. Ensure sufficient contrast.
- Support **Bangla and English** and test long text and different fonts.
- Build a "skip to content" link and meaningful page titles for each route.

10.2 Backend and database

Pagination and efficient API responses

- Never return an unbounded list. Always paginate with a maximum page size (for example 100).
- **Offset pagination** (`LIMIT 20 OFFSET 40`) is simple and fine for the MVP. **Keyset pagination** (`WHERE created_at < $last`) is faster for very large tables and can be adopted later.
- Return only the fields the screen needs: a list endpoint returns summary fields, the detail endpoint returns more. Avoid `SELECT *`.
- Do aggregation in SQL (`GROUP BY`), not by fetching rows and counting in JavaScript.
- Avoid the **N+1 query** pattern (one query for the list, then one query per row). Use a join or one query with `WHERE id = ANY($1)`.
- Enable response compression (gzip or Brotli) at the server or reverse proxy.

Database indexes and query optimization

Query it serves	Suggested index
Coordinator list: requests by status and urgency	`(status, urgency, created_at DESC)`
Filter by administrative area	`(admin_area_id, status)`
Reporter sees own requests	`(reporter_id, created_at DESC)`
Unread notification badge	`(user_id, read_at, created_at DESC)`
Resources by organization and type	`(organization_id, resource_type)`
Timeline for one record	`(entity_type, entity_id, changed_at)`
Duplicate candidate search	`(category, admin_area_id, created_at)` plus optional `pg_trgm` index on description
Map markers by area	`(admin_area_id, status)` now; spatial (PostGIS GiST) index later

- Use `EXPLAIN ANALYZE` to see how PostgreSQL executes a slow query and whether it uses an index.
- Add indexes for real, frequent queries. Every index slows writes slightly and uses space.
- Keep transactions short; do not call external services while holding row locks.

Connection pooling

Opening a database connection is expensive. A **pool** keeps a small number of connections open and lends them to requests. Create the pool once at startup, keep its maximum small (for example 10 to 20, and never more than the database allows), always **release** connections used in transactions, and set timeouts. On hosted databases with a low connection limit, use the provider's connection pooler.

10.3 Map performance

- Request only markers inside the **visible bounding box** and current filters; refetch when the map stops moving (debounced).
- Cluster markers on the client (for example Leaflet markercluster) so thousands of points become a few numbered circles.
- Return a minimal marker payload (id, position, urgency, category, status) and load details only when a marker is clicked.
- If data becomes very large, aggregate into clusters on the server (grid or area counts) and return those at low zoom levels.
- Show approximate positions for privacy, which also reduces precision overhead.

10.4 Performance checklist

Check	Target
Every list endpoint paginated with a maximum page size	Yes
Indexes exist for the main filters and sorts	Yes (verified with EXPLAIN ANALYZE)
No N+1 queries on list pages	Yes
Routes code-split; map and charts loaded lazily	Yes
Skeletons, error and empty states on every data screen	Yes
Forms usable by keyboard and screen reader; labels present	Yes
Usable on a small phone and slow 3G network (tested)	Yes

11. Testing

Tests prove that the important rules keep working as the code changes. For FloodRelief the highest-value tests protect **permissions**, **status workflows** and **inventory integrity**. The original PDF recommends unit, integration and end-to-end tests with a focus on permissions, invalid input, duplicate submissions and inventory edge cases.

11.1 Types of tests and recommended tools

Type	What it checks	Recommended tools (PERN)	FloodRelief example
Unit tests	One function in isolation, with no web server or database.	Vitest (or Jest)	canTransition("verified", "submitted") returns false; urgency scoring; Zod schemas.
Backend API integration tests	Real HTTP requests against the Express app and a test database.	Vitest or Jest + Supertest + a real PostgreSQL test database (Docker)	POST /api/requests creates a row and returns 201; invalid body returns 422.
Database transaction tests	Behaviour under rollback and concurrency.	Same as above, with Promise.all for concurrent calls	Two simultaneous allocations of the same stock: only one succeeds.
Frontend component tests	Components render and behave correctly.	Vitest + React Testing Library, MSW to mock the API	Request form shows validation errors; badge shows the right urgency.
End-to-end tests	Complete user journeys in a real browser.	Playwright	Reporter submits, coordinator verifies and allocates, volunteer completes.
Authorization and security tests	Forbidden actions are really forbidden.	Supertest, Playwright, optional OWASP ZAP baseline scan	Volunteer calling the verification endpoint receives 403.

RECOMMENDATION (not in the original document)
Test against a **real PostgreSQL** instance (for example in Docker) rather than mocking the database. Mocks cannot reveal constraint, transaction or locking problems, which are exactly what matters here. Reset the database with migrations and seed data before each test run or each test file.

11.2 Unit test example: status transitions

```
import { describe, it, expect } from "vitest";
import { canTransition } from "../src/modules/requests/requests.rules";

describe("request status transitions", () => {
  it("allows submitted -> verified", () => {
    expect(canTransition("submitted", "verified")).toBe(true);
  });
  it("does not allow fulfilled -> rejected", () => {
    expect(canTransition("fulfilled", "rejected")).toBe(false);
  });
});
```

11.3 Integration test example: authorization

```
import request from "supertest";
import { app } from "../src/app";
import { loginAs, createRequestAs } from "./helpers";

describe("verification permissions", () => {
  it("rejects anonymous callers with 401", async () => {
    await request(app).patch("/api/requests/" + id + "/verification")
      .send({ decision: "verified" }).expect(401);
  });
  it("rejects a volunteer with 403", async () => {
    const volunteer = await loginAs("volunteer");
    await volunteer.patch("/api/requests/" + id + "/verification")
      .send({ decision: "verified" }).expect(403);
  });
  it("does not let a reporter read someone else's request", async () => {
    const other = await loginAs("reporter", "other@example.test");
    await other.get("/api/requests/" + id).expect(404);
  });
  it("ignores a role field sent during registration", async () => {
    const res = await request(app).post("/api/auth/register")
      .send({ email: "x@example.test", password: "a-long-password", role: "admin" });
    expect(res.status).toBe(422);            // strict schema rejects the unknown field
  });
});
```

11.4 Database transaction test: preventing over-allocation

This is the most important test in the project. It proves that two coordinators cannot reserve the same stock.

```
it("never allocates more than the available stock under concurrency", async () => {
  // 60 bottles available
  const stock = await createResource({ type: "drinking_water", total: 60 });
  const [r1, r2] = await Promise.all([
    coordinatorA.post("/api/assignments").send(alloc(request1, stock.id, 60)),
    coordinatorB.post("/api/assignments").send(alloc(request2, stock.id, 40)),
  ]);
  const statuses = [r1.status, r2.status].sort();
  expect(statuses).toEqual([201, 409]);         // exactly one succeeds, one conflicts

  const after = await getResource(stock.id);
  expect(after.available_quantity).toBeGreaterThanOrEqual(0);
  const allocated = await sumAllocatedItems(stock.id);
  expect(after.total_quantity - after.available_quantity).toBe(allocated);
});

it("rolls back everything if the second resource is insufficient", async () => {
  // allocate 60 from A and 40 from B where B only has 10
  const res = await coordinator.post("/api/assignments").send(allocAAndB);
  expect(res.status).toBe(409);
  expect((await getResource(A.id)).available_quantity).toBe(60);   // unchanged
  expect(await countAssignmentsFor(request1.id)).toBe(0);          // no partial rows
});
```

> **NOTE**
> In the first test above, one of the two allocations must fail whichever transaction gets the lock first (60 + 40 exceeds 60). That is exactly the behaviour to prove. Also test the database CHECK constraint directly by trying to set available_quantity below zero in SQL and expecting an error.

11.5 Frontend tests

```
test("shows validation messages and does not submit an empty request form", async () => {
  render(<RequestForm />);
  await userEvent.click(screen.getByRole("button", { name: /submit request/i }));
  expect(await screen.findByText(/description is required/i)).toBeInTheDocument();
  expect(submitSpy).not.toHaveBeenCalled();
});
```

11.6 End-to-end test: the demo scenario

```
test("water request flows from report to completed delivery", async ({ browser }) => {
  // 1. Reporter submits an urgent water request
  // 2. Coordinator logs in, opens it, verifies it
  // 3. Org member has added 60 bottles (seed data)
  // 4. Coordinator allocates 60 bottles and creates a task for a volunteer
  // 5. Volunteer logs in, accepts, completes the task with proof
  // 6. Coordinator accepts proof; dashboard shows 1 completed delivery
  // 7. Request timeline shows every status change in order
});
```

11.7 Security-focused tests

Test	Expected result
No cookie on any protected route	401
Each role calling each other role's endpoint (permission matrix, table-driven test)	403 or 404
Changing an id in the URL to another user's record	404 or 403
SQL injection string in search or email fields	Treated as plain text; no error leakage
HTML or script tags in a description	Stored as text; rendered escaped in the UI
Excess fields (status, role, reporterId) in request bodies	422 and the values are ignored
Eleven login attempts in a short window	429
Upload a non-image, an oversized image and a file with a fake extension	Rejected with a clear message
Suspended user with an existing session	Access denied
Response bodies never contain password_hash or other users' phone numbers	Verified in tests

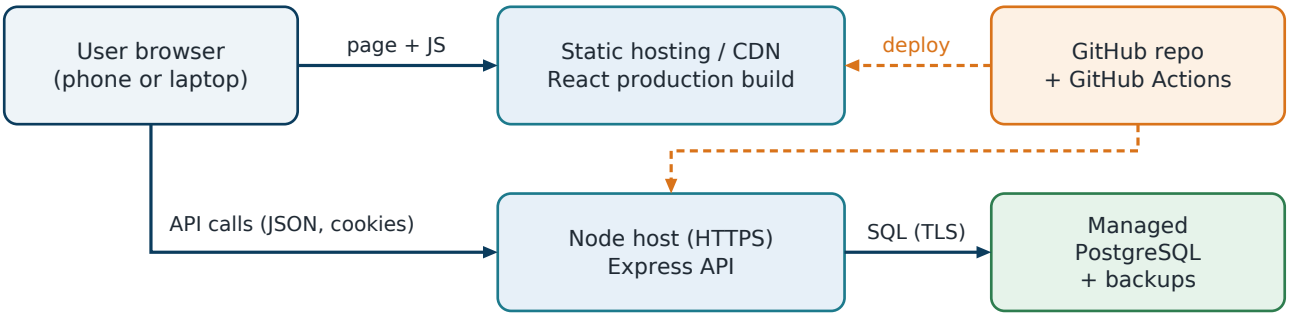
11.8 What to run and when

- **On every commit locally:** lint, type-check, unit tests.
- **On every pull request (GitHub Actions):** type-check, unit, integration (with a PostgreSQL service container), frontend tests.
- **Before release:** end-to-end suite and a manual security review (chapter 9 checklist).

12. Deployment

Deployment means making the application available on the internet. For a portfolio project, keep it simple: three pieces (frontend, backend, database), HTTPS, backups, and an automated pipeline. Avoid infrastructure you cannot explain.

12.1 Deployment architecture



Environment variables (DATABASE_URL, JWT/session secrets, CORS origin) are set on the Node host, never in React code.

Figure 12.1: A simple beginner-friendly deployment layout.

12.2 The three parts

Part	What is deployed	Typical beginner-friendly options	Notes
React frontend	The static files produced by `npm run build` (HTML, JavaScript, CSS).	Static hosting with a CDN (for example Vercel, Netlify, Cloudflare Pages, GitHub Pages).	Set the API base URL at build time. No secrets in the build.
Express / Node backend	The compiled TypeScript run by Node.js.	A Node-capable host (for example Render, Railway, Fly.io) or a small VPS; optionally a Docker image.	Set environment variables on the host. Provide a `/health` endpoint. HTTPS is usually included.
PostgreSQL database	A managed PostgreSQL instance.	A managed provider (for example Neon, Supabase, Render or Railway Postgres) or Postgres in Docker on a VPS.	Check provider support for PostGIS and pg_trgm, backup features and connection limits. Do not expose the database to the whole internet.

NOTE
Provider offerings, free tiers and prices change often. Check each provider's current documentation and limits before choosing. The architecture in this guide does not depend on a specific provider.

12.3 Development versus production environments

Aspect	Development (your computer)	Production (the internet)
Frontend	Vite dev server with hot reload.	Static build served by a CDN.
Backend	ts-node-dev or tsx with auto restart.	Compiled JavaScript run by Node.
Database	Local PostgreSQL (Docker) with synthetic seed data.	Managed PostgreSQL with backups. Only synthetic data for the portfolio demo.
Environment variables	.env file (ignored by Git).	Set in the host's secret manager.
Cookies	Secure flag off on http://localhost.	Secure, HttpOnly, SameSite on HTTPS.
Errors	Detailed errors for debugging.	Generic messages to users; details in logs.
CORS	Allows http://localhost:5173.	Allows only the real frontend domain.

12.4 Database migrations in deployment

- Migrations live in the repository and run in order. The same files run in development, CI and production.
- Apply migrations as a deployment step **before** the new backend starts.
- Prefer **backward-compatible changes** (add a column, then later remove the old one) so a rollback of the code is possible.
- Never edit an already-applied migration; create a new one.
- Take a backup before risky migrations, and explain the impact before running anything destructive.

12.5 Backups

- Enable **automated daily backups** in the managed database; know the retention period.
- Optionally script `pg_dump` for an additional copy stored somewhere separate.
- **Test a restore** at least once. A backup that has never been restored is only a hope.
- Because the demo uses synthetic data, losing it is not serious, but practising backup and restore is valuable for the portfolio and for any later real pilot.

12.6 HTTPS

HTTPS encrypts traffic between the browser and the server so passwords and cookies cannot be read on the network. Most hosting platforms provide free certificates automatically. On your own VPS, use a reverse proxy such as Caddy (automatic certificates) or Nginx with Let's Encrypt. Redirect HTTP to HTTPS and enable the `Secure` cookie flag.

12.7 Basic logging and monitoring

- Use structured logging (for example **pino**) with a request id for each request. Log method, path, status and duration. **Never log passwords, tokens, session ids or full personal details.**

- Provide GET `/health` that checks the server and a simple database query.
- Use an uptime monitor that calls the health endpoint and alerts you when it fails.
- Optionally add error tracking (for example Sentry) once the application is public. Keep this optional.

12.8 Git and GitHub

- One repository (monorepo) with `frontend/` and `backend/` folders is simplest for a solo project.
- Use a `main` branch that always works and short-lived feature branches (`feature/request-form`).
- Make small, meaningful commits and open pull requests, even when working alone, to practise the workflow.
- Maintain a good `.gitignore` (`node_modules`, `.env`, build output, uploads).
- Write a README: what it is, the disclaimer, how to run it, architecture diagram, and screenshots.

12.9 CI/CD with GitHub Actions

Continuous Integration (CI) automatically runs checks on every push. **Continuous Delivery/Deployment (CD)** automatically ships a passing version. Start with CI only; add automatic deploys later if desired.

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_PASSWORD: postgres, POSTGRES_DB: floodrelief_test }
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U postgres" --health-interval 5s
          --health-timeout 5s --health-retries 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci --prefix backend && npm ci --prefix frontend
      - run: npm run typecheck --prefix backend && npm run typecheck --prefix frontend
      - run: npm run migrate --prefix backend
        env: { DATABASE_URL: "postgres://postgres:postgres@localhost/floodrelief_test" }
      - run: npm test --prefix backend
        env: { DATABASE_URL: "postgres://postgres:postgres@localhost/floodrelief_test" }
      - run: npm test --prefix frontend
```

12.10 Docker (use it where it helps)

Docker packages software and its dependencies into containers that run the same way everywhere. For FloodRelief, the most valuable use is running **PostgreSQL locally** without installing it manually. A Dockerfile for the backend is optional and useful if your host accepts containers. Do not add Kubernetes or orchestration.

```
# docker-compose.yml  (local development database)
services:
  db:
    image: postgres:16          # switch to a PostGIS image when you add PostGIS
    environment:
      POSTGRES_USER: floodrelief
      POSTGRES_PASSWORD: devpassword
      POSTGRES_DB: floodrelief
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
volumes:
  pgdata:
```

13. Development Roadmap

This roadmap follows the thirteen phases you specified. It is ordered so that each phase builds on the previous one and you always have something working. The effort estimates are rough guesses for a beginner working part-time and should be adjusted to your own pace.

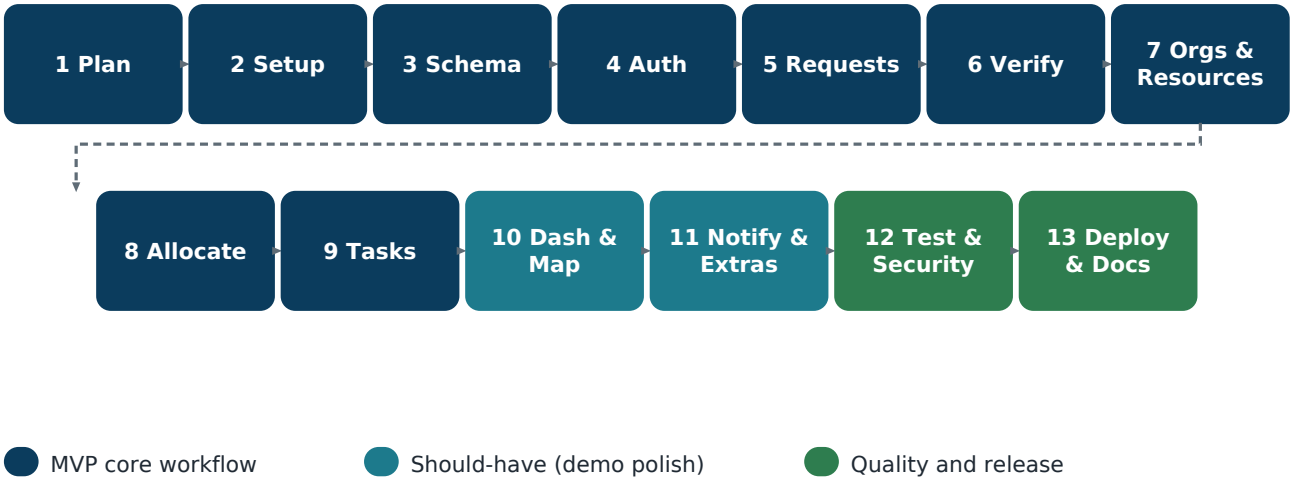


Figure 13.1: The thirteen phases. Navy = MVP core workflow, teal = should-have polish, green = quality and release.

Phase 1. Requirements and planning

What to build	Confirm MVP scope, roles and permission matrix, request status flow, the demo scenario, and low-fidelity wireframes (paper or a drawing tool). Choose a project name, repository name and license. Write the disclaimer text.
Why at this stage	Cheap to change now, expensive later. Clear scope prevents the common failure of an unfinished giant project.
Technologies	Markdown, a drawing or wireframe tool, this document.
Expected result	A one-page MVP scope, a list of user journeys, wireframes for 5 to 7 key screens.
Done when	You can explain the demo scenario end to end and state what is NOT included.
Rough effort	About 1 week.

Phase 2. Project setup

What to build	Install Node.js (LTS), Git, PostgreSQL (or Docker), a code editor. Create the monorepo with `backend/` and `frontend/`, TypeScript strict mode, ESLint and Prettier, a hello-world Express server with a `/health` route, a Vite + React app that calls it, and a GitHub repository with the first CI workflow.
Why at this stage	A working skeleton gives fast feedback and a place to put everything else.
Technologies	Node.js, npm, TypeScript, Express, Vite, React, Git, GitHub Actions, Docker (optional).
Expected result	Both apps run locally and the frontend displays a message fetched from the backend.
Done when	A new clone of the repository can be set up following the README in under 15 minutes.
Rough effort	About 1 week.

Phase 3. Database schema and migrations

What to build	Choose the migration tool, create the migrations for users, organizations, organization_members, admin_areas, assistance_requests, request_reviews, resources, assignments, assignment_items, volunteer_tasks, status_history, notifications and the indexes and constraints. Write a seed script with synthetic data. Create the `pool` module and a transaction helper.
Why at this stage	Everything else reads and writes these tables. Constraints added now catch many later bugs.
Technologies	PostgreSQL, SQL, node-pg-migrate (or your choice), pg.
Expected result	The database can be created from scratch with one command and filled with demo data.
Done when	Migrations run on an empty database; constraints reject invalid sample inserts; seed data loads.
Rough effort	About 1 to 2 weeks.

Phase 4. Authentication and authorization

What to build	Register, login, logout, `GET /api/auth/me`, password hashing, secure cookie sessions, `requireAuth` and `requireRole`, Zod validation middleware, central error handler, rate limiting on auth, Helmet and CORS. Frontend: login and register pages, protected routes, auth context, role-based navigation.
Why at this stage	Every later feature depends on knowing who the caller is and what they may do. Building security first is easier than adding it later.
Technologies	Argon2id or bcrypt, express sessions or JWT cookie, Zod, React Router, React Hook Form, TanStack Query.
Expected result	Users can register, sign in and see only the pages for their role; unauthorized API calls return 401 or 403.
Done when	Tests prove: anonymous gets 401, wrong role gets 403, role cannot be chosen at registration, passwords are never stored in plain text.
Rough effort	About 2 weeks.

Phase 5. Assistance request workflow

What to build	Request form (with cascading area selection), `POST /api/requests`, list with filters and pagination, detail page, reporter's "my requests", status history writing, basic duplicate suggestion query (optional placeholder).
Why at this stage	The first real feature of the platform; it exercises validation, ownership checks, pagination and the status history pattern that later phases reuse.
Technologies	Express routes, services, repositories, Zod, React forms, TanStack Query, pagination.
Expected result	A reporter submits a request and sees it in their list; a coordinator sees it in the area list.
Done when	Reporter cannot see others' requests; validation errors display; list is paginated and filtered.
Rough effort	About 2 weeks.

Phase 6. Verification workflow

What to build	Review panel, PATCH `/api/requests/:id/verification`, allowed status transitions in one rules module, request_reviews, status_history, notifications placeholders, rejection and needs-info notes, duplicate confirm or dismiss.
Why at this stage	Introduces the first coordinator-only action and the transaction pattern (status + review + history in one commit).
Technologies	Express, PostgreSQL transactions, RBAC, React.
Expected result	A coordinator can verify, reject or request information, and the reporter sees the new status.
Done when	Invalid transitions are refused with 409; every decision creates review and history rows atomically.
Rough effort	About 1 week.

Phase 7. Organization and resource management

What to build	Organization registration and approval, membership management, resource CRUD, quantity adjustment endpoint with ledger rows, inventory table with low-stock indication.
Why at this stage	Allocation (next phase) needs trustworthy organizations and stock.
Technologies	Express, PostgreSQL constraints, React tables and dialogs.
Expected result	Organization members maintain stock; admins approve organizations; coordinators view all stock.
Done when	CHECK constraints reject negative stock; members cannot touch another organization's resources; ledger explains every number.
Rough effort	About 2 weeks.

Phase 8. Resource allocation

What to build	Allocation screen, POST `/api/assignments` with a single transaction using row locks, inventory_movements, cancellation with release of stock, notifications to the reporter and organizations.
Why at this stage	It is the technical heart of the project, so build it once the surrounding data and permissions are stable.
Technologies	PostgreSQL transactions and FOR UPDATE, pg client transactions, Vitest.
Expected result	The 100-bottle example (60 from A, 40 from B) works, and concurrent attempts cannot over-allocate.
Done when	The concurrency test passes repeatedly; rollbacks leave no partial rows.
Rough effort	About 2 weeks.

Phase 9. Volunteer task management

What to build	Task creation from an assignment, volunteer "my tasks" page, accept, decline, start, complete, unable to complete, optional teams, proof of delivery form and coordinator review, assignment and request status progression.
Why at this stage	Closes the loop from need to delivery. Proof of delivery depends on tasks existing.
Technologies	Express, multer and sharp (for optional photo), React mobile-friendly UI.
Expected result	The full demo scenario works from report to completed, reviewed delivery.
Done when	Volunteers can only touch own tasks; uploads are validated and stored privately; statuses and history update correctly.
Rough effort	About 2 weeks.

Phase 10. Dashboard and map

What to build	Dashboard summary endpoints and screens, by-area counts, interactive map with bounding-box loading, status and urgency filters, location search, marker clustering, approximate coordinates for privacy.
Why at this stage	These are read-only views over data that now exists, so they are safer and easier to build after the workflow.
Technologies	SQL aggregation, Leaflet and react-leaflet, marker clustering, TanStack Query.
Expected result	Coordinators can understand the situation at a glance and explore requests on a map.
Done when	The map stays smooth with a few thousand synthetic requests; numbers match the database.
Rough effort	About 2 weeks.

Phase 11. Notifications and additional features

What to build	Notification bell with unread count and list, polling, Bangla and English UI, duplicate suggestion improvements, audit log viewer for administrators.
Why at this stage	Polish features that improve usability, added once the core workflow is stable.
Technologies	react-i18next, PostgreSQL pg_trgm (optional), React.
Expected result	The app can be used in Bangla or English, and users are informed of important changes.
Done when	All user-facing text comes from translation files; notification counts are correct.
Rough effort	About 2 weeks.

Phase 12. Testing and security review

What to build	Fill gaps in unit, integration, transaction and end-to-end tests. Walk through the security checklist from chapter 9, run dependency audits, review error messages and logs, test accessibility and mobile layouts, test performance with a larger synthetic dataset.
Why at this stage	Before release you need evidence that the important rules hold. Parts of this happen continuously, but this phase is a focused review.
Technologies	Vitest, Supertest, Playwright, npm audit, EXPLAIN ANALYZE, browser accessibility tools.
Expected result	A documented list of what is tested and what was fixed.
Done when	CI is green; permission matrix is tested; checklist items are all confirmed.
Rough effort	About 2 weeks.

Phase 13. Deployment and documentation

What to build	Deploy frontend, backend and database; configure environment variables, HTTPS, backups, logging and health checks; write the README, API documentation (OpenAPI/Swagger is optional), architecture diagram, screenshots and a short demo video; add the demo disclaimer everywhere it belongs.
Why at this stage	Shipping and explaining the project is what turns code into a portfolio piece.
Technologies	Hosting providers, GitHub Actions, Docker (optional), Markdown.
Expected result	A public demo using synthetic data with clear documentation.
Done when	A stranger can open the demo, log in with sample accounts, follow the scenario and understand the design from the README.
Rough effort	About 1 to 2 weeks.

RECOMMENDATION (not in the original document)
At 8 to 10 hours per week the sequence above is roughly five to six months. If you are short on time, stop after phase 9 plus a basic phase 12 and 13: that is a complete, demonstrable MVP.

13.1 MVP versus later enhancements

Priority	Features	Roadmap phases
Must have (MVP)	Login and roles, request submission, request list and detail, verification, organizations and resource records, allocation, assignments and volunteer tasks, status updates, status history.	2 to 9
Should have	Map view and filters, dashboard summary, notifications, audit log viewer, duplicate suggestions, Bangla and English, proof of delivery, location hierarchy.	5, 10, 11
Quality	Automated tests, security review, accessibility, performance checks, documentation, deployment.	12 to 13
Later (not MVP)	SMS notifications, advanced geographic matching with PostGIS, advanced analytics, offline-first mobile support, advanced duplicate detection, extra reporting and exports.	After release

ORIGINAL REQUIREMENT
The MVP definition follows the original PDF: the must-have list proves the complete request-to-assignment workflow; map, filters, dashboard, activity history and notifications are should-have; advanced matching, SMS, offline-first mobile, analytics, multilingual UI and automated duplicate suggestions are later. In this document, Bangla/English and proof of delivery are included as should-have features because you listed them as differentiators; move them to "later" if time is limited.

14. Recommended Project Structure

FloodRelief should be a **modular monolith**: one backend application (and one frontend application) whose code is organized into clearly separated modules, deployed as a single unit. This gives the benefits of organization without the cost of microservices. Each backend module (requests, resources, assignments...) owns its routes, controllers, services, repository and validation schemas, and talks to other modules through their services.

14.1 Repository layout

```
floodrelief/
  README.md                  what the project is, disclaimer, how to run
  docker-compose.yml         local PostgreSQL
  .github/workflows/ci.yml   automated checks
  docs/                      this PDF, decisions, diagrams
  backend/
    package.json  tsconfig.json  .env.example
    migrations/              versioned SQL migrations
    src/
      server.ts              starts the HTTP server
      app.ts                 builds the Express app and middleware chain
      config/env.ts          reads and validates environment variables
      db/                    pool.ts, transaction.ts, seeds/
      middleware/            requireAuth, requireRole, validate, rateLimit,
                             csrf, errorHandler
      modules/
        auth/  users/  organizations/  areas/  requests/  verification/
        resources/  assignments/  tasks/  notifications/  dashboard/
          (each module: *.routes.ts  *.controller.ts  *.service.ts
                        *.repo.ts    *.schemas.ts     *.rules.ts)
      shared/                errors.ts, logger.ts, pagination.ts, audit.ts, types.ts
    tests/                   unit/  integration/  helpers/
  frontend/
    package.json  vite.config.ts  index.html  tsconfig.json
    src/
      main.tsx  App.tsx
      routes/                route table, ProtectedRoute
      api/                   client.ts (fetch/axios setup), one file per backend area
      features/              auth/ requests/ verification/ resources/ assignments/
                             tasks/ notifications/ dashboard/ map/
      components/            ui/ (buttons, inputs, table, badge)  layout/ (navbar, sidebar)
      hooks/                 shared custom hooks
      i18n/                  en.json  bn.json  index.ts
      lib/                   formatters, constants, helpers
      types/                 shared TypeScript types
      styles/
    tests/                   component tests and e2e/ (Playwright)
```

14.2 Purpose of the important backend folders

Folder / file	Purpose	Must NOT contain
server.ts	Starts listening on a port. Nothing else.	Routes or business logic.
app.ts	Creates the Express app, registers middleware and module routers, registers the error handler.	Business rules.
config/	Loads and validates environment variables once.	Secrets in code.
db/	Connection pool, transaction helper, seed scripts.	Feature-specific SQL.
middleware/	Reusable request filters: authentication, role checks, validation, rate limits, CSRF, errors.	Feature-specific business rules.
modules/<name>/	Everything about one feature area. Keeps related files together.	Imports of another module's repository (go through its service).
migrations/	Versioned database structure changes.	Application code.
shared/	Small utilities used by several modules.	A dumping ground for unrelated code.
tests/	Automated tests.	Real personal data.

14.3 The separation of responsibilities

Part	Responsibility	Example in the requests module
Route	Declares URL and method, attaches middleware and the controller.	`router.post("/", requireAuth, validate(createSchema), ctrl.create)`
Controller	Translates HTTP to a function call and back. Thin.	Reads `req.body`, calls the service, sends `201` with JSON.
Service	Business rules, permission and ownership checks, transactions, audit and history writing.	`createRequest`, `verifyRequest`, status transition rules.
Repository	All SQL for this module; returns plain objects.	`insertRequest`, `findById`, `listByFilters`.
Validation schema	Zod description of the allowed input; also yields TypeScript types.	`createRequestSchema`, `listQuerySchema`.
Middleware	Cross-cutting checks that run before the controller.	`requireAuth`, `requireRole`, `validate`.
Rules file (optional)	Pure functions with no I/O that are easy to unit test.	`canTransition(from, to)`.

The golden rule: a request flows *route* → *middleware* → *controller* → *service* → *repository* → *database* and the response flows back. A layer only calls the layer directly below it. This keeps the code easy to read, test and change.

14.4 Purpose of the important frontend folders

Folder	Purpose
api/	The only place that knows URLs and HTTP details. One function per endpoint, typed with request and response types.
features/	Pages, components, hooks and queries grouped by feature (for example everything for requests). Easier to navigate than grouping by file type.
components/ui	Small, reusable, feature-independent building blocks: buttons, inputs, modals, tables, badges.
components/layout	App shell: navigation, sidebar, page container.
routes/	Route table and protected route logic (redirects by authentication and role).
hooks/	Reusable hooks, for example `useDebounce` or `useAuth`.
i18n/	Translation files for Bangla and English.
lib/, types/	Helpers (date and number formatting) and shared TypeScript types.

RECOMMENDATION (not in the original document)
Sharing types and Zod schemas between frontend and backend is attractive. A simple way is an npm workspace package called `shared/`. Start without it (copy the few schemas you need) and extract it once duplication becomes annoying.

15. Things to Avoid in the Initial Version

The MVP should use the smallest architecture that delivers the core workflow. Each item below may be reconsidered later **if a concrete need appears**.

> **PRINCIPLE (from the original document)**
> A smaller, complete, well-tested product is stronger than a huge unfinished feature list. The portfolio value comes from the quality of the implementation and the clarity of the explanation, not from the number of tools.

Avoid at first	Why	Reconsider when
Microservices	They add network calls, deployment complexity and distributed transactions. Inventory allocation needs one database transaction, which is natural in a monolith.	Separate teams or very different scaling needs make a measured, specific split worthwhile.
Kubernetes	It solves problems of running many services at scale, which one backend and a managed database do not have.	You operate many services and have a real orchestration need.
Redis	Not needed to be fast at this scale. It adds another service, cache invalidation and failure modes. PostgreSQL with indexes and pagination is sufficient.	A specific, measured need appears (slow repeated reads, shared rate limits, job queues).
AI-based urgency decisions	Wrong or biased urgency could misdirect help and is hard to explain and audit. The original PDF says not to add it to the first version.	After a validated pilot, as a suggestion tool that a human reviews, with testing and transparency.
Automatic relief allocation	Allocation affects real people and scarce stock. Humans must approve it, and the instruction for this project requires human approval in the MVP.	Only as suggestions that a coordinator confirms.
Advanced GIS	Full spatial analysis, routing and boundary geometry are large topics. Latitude and longitude plus the area hierarchy are enough to prove the idea.	Distance or boundary queries become real requirements; then adopt PostGIS.
Native mobile apps	Two more platforms, store publishing, and duplicated work. A responsive web app already works on phones.	The web workflow is stable and there is a clear need such as offline use; consider a PWA first.
Unnecessarily complex analytics	Charts without meaningful, real data mislead. Simple counts from real records are honest and useful.	There is enough data and a clear question to answer.
Integrations with real emergency or government systems	They need authorization, agreements and security reviews far beyond a portfolio project.	A formal partnership exists.
Real personal data in demos	It creates privacy and safety risks and possible legal issues.	Never in a public demo; only in a formally approved pilot.

16. Privacy and Responsible Use

Disaster-related information is sensitive. A real deployment would need careful legal, ethical and security work. This project is a prototype and must be built and presented accordingly.

16.1 Use synthetic demo data

- Real flood victims' data can expose them to harm (theft, harassment, targeting) and creates legal and ethical obligations.
- All people, phone numbers, locations, organizations and statistics in the demo must be clearly fictional and generated by a seed script.
- Use reserved example domains (`example.test`) and obviously fake phone numbers.
- Never copy real social media posts or victim reports into the demo.
- Do not present simulated numbers as real disaster statistics; label dashboards "synthetic demo data".

16.2 Do not expose exact household locations publicly

An exact location of a household that needs help, shown to everyone, can attract criminals or cause harm, especially for vulnerable people. FloodRelief should store exact coordinates privately, show only approximate positions (for example rounded or at area level) in broader views, and reveal the exact position only to authorized coordinators and to the volunteer assigned after they accept the task.

16.3 Restrict phone numbers and personal details

- Collect **only what is needed** (data minimization). Contact details are optional where possible.
- Phone numbers are visible only to coordinators who need them and to the assigned volunteer for the task.
- Never show contact details on public pages, in maps or in list views.
- Apply field-level filtering in the backend DTOs: the API response for a role simply does not contain fields that the role must not see.
- Do not put personal details in logs, URLs, notification text or error messages.
- Define a retention rule: how long data and photos are kept, and how they are deleted.

16.4 Do not present it as an official emergency service

- Do not claim that reporting through the platform guarantees help, rescue or response times.
- Do not imply affiliation with a government body or relief agency.
- Show the disclaimer on the landing page, on the request form and in the README.
- Do not claim real-world impact until a pilot has been run with permission, appropriate partners and privacy and safety controls (original PDF).

16.5 Why human verification is important

- Reports can be mistaken, duplicated, exaggerated or malicious. Resources are limited and every wrong allocation takes help from someone else.
- A human verifier understands local context that software cannot (road conditions, who is reliable).
- Human decisions create accountability: each verification and allocation names a responsible person and is recorded.
- Software that decides alone can fail silently. Human approval makes failures visible and correctable.
- Therefore in this project: **verification is manual**, **allocation is human-approved**, and duplicate detection **only suggests**.

16.6 Suggested demo disclaimer

> **SUGGESTED DEMO DISCLAIMER (English)**
> **FloodRelief is a demonstration and portfolio project.** All people, locations, organizations, requests, resources and numbers shown here are synthetic and fictional. This site is not an emergency service and is not affiliated with any government agency or relief organization. It does not dispatch help and cannot guarantee any response.
> **If you or someone else is in danger, contact your official local emergency services immediately** (for example, the national emergency helpline in your country; verify the correct number before publishing it). Please do not enter real personal information into this demo.

Place this text, shortened if needed, in the footer of every page, on the request form, and at the top of the README. Provide a Bangla translation of the same text when the Bangla interface is added.

17. Glossary of Technical Terms

Short, beginner-friendly definitions of the terms used in this document.

Term	Meaning
ACID	Atomicity, Consistency, Isolation, Durability: the guarantees of reliable database transactions.
API	Application Programming Interface. A defined way for programs (here React and Express) to communicate.
Argon2id	A modern, deliberately slow password-hashing algorithm.
Audit log	A tamper-resistant record of who did what and when.
Authentication	Checking who a user is.
Authorization	Checking what an authenticated user is allowed to do.
Backend	The server-side part that applies rules, checks permissions and accesses the database.
bcrypt	An older but still accepted password-hashing algorithm.
Bounding box	The rectangle of map coordinates currently visible; used to load only nearby markers.
Cache	A faster temporary store used to avoid repeating work.
CHECK constraint	A database rule that every row must satisfy, for example quantity greater than zero.
CI/CD	Continuous Integration / Continuous Delivery: automatic checks and releases when code changes.
Clustering (map)	Merging nearby markers into one numbered circle when zoomed out.
Component	A reusable React building block that returns part of the interface.
Connection pool	A set of reusable database connections kept open by the backend.
Constraint	A rule enforced by the database (NOT NULL, UNIQUE, FOREIGN KEY, CHECK).
Controller	The backend function that handles an HTTP request and sends the response.
Cookie	A small piece of data the browser stores and sends back to the server automatically.
CORS	Cross-Origin Resource Sharing: rules about which websites may call an API from a browser.
CRUD	Create, Read, Update, Delete: the basic data operations.
CSRF	Cross-Site Request Forgery: tricking a signed-in browser into sending an unwanted request.
Database	Organized, persistent storage for application data.
Deadlock	Two transactions each waiting for a lock the other holds.
Docker	A tool that packages software into containers that run the same everywhere.
E2E test	End-to-end test: a test of a complete user journey through a real browser.
Endpoint	A URL plus an HTTP method that performs a defined API operation.

Term	Meaning
Environment variable	A configuration value (often a secret) supplied to the program from outside the code.
Express.js	A minimal web framework for Node.js.
Foreign key (FK)	A column that refers to the primary key of another table.
Framework	A structured set of tools and conventions for building an application.
Frontend	The part users see and interact with in the browser.
Hash	A one-way scrambled value; used to store passwords safely.
Helmet	Express middleware that sets protective security headers.
Hook (React)	A function such as useState that lets a component use React features.
HTTP	The protocol browsers and servers use to exchange requests and responses.
HTTPS	HTTP encrypted with TLS.
Index	A database structure that speeds up finding rows, like a book index.
JSON	A text format for structured data.
JWT	JSON Web Token: a signed token carrying claims about a user.
Library	Reusable code your application calls when needed.
Migration	A versioned script that changes the database structure.
Middleware	A function that runs before the controller and can inspect, change or stop the request.
Modular monolith	One deployable application organized into separate internal modules.
MVP	Minimum Viable Product: the smallest version that proves the main idea.
Node.js	A runtime that runs JavaScript outside the browser.
npm	The package manager for Node.js and frontend libraries.
ORM	Object-Relational Mapper: maps code objects to database rows.
Pagination	Returning data in pages instead of everything at once.
Parameterized query	A SQL query where values are sent separately from the SQL text; prevents SQL injection.
PERN	PostgreSQL, Express.js, React, Node.js.
PostGIS	A PostgreSQL extension for geographic data and spatial queries.
PostgreSQL	An open-source relational database.
Primary key (PK)	A column that uniquely identifies each row in a table.
Props	Inputs passed from a parent React component to a child.
Rate limiting	Restricting how many requests a client can make in a time window.
RBAC	Role-Based Access Control: permissions granted by role.
React	A JavaScript library for building user interfaces from components.
Repository	The module that contains the SQL and talks to the database.
REST	A common style of designing web APIs around resources and HTTP methods.

Term	Meaning
Route	A mapping from an HTTP method and URL to a handler.
Seed data	Sample data loaded into a database for development and demos.
Service	The layer containing business rules, independent of HTTP.
Session	Server-side record that remembers a signed-in user between requests.
SPA	Single-page application: loads once and updates the page without full reloads.
SQL	Structured Query Language: the language used to query and change relational databases.
SQL injection	An attack that inserts malicious SQL through user input.
State (React)	Data a component remembers that triggers a re-render when it changes.
Status code	A number in an HTTP response showing the outcome (200, 403, 409...).
Synthetic data	Fake data created for demos and tests.
Transaction	A group of database operations that all succeed or all fail together.
TypeScript	JavaScript with static types, compiled to plain JavaScript.
Vite	A fast build tool and development server for frontend projects.
XSS	Cross-Site Scripting: injecting malicious scripts into a web page.
Zod	A TypeScript library for describing and validating data shapes at run time.

18. Project Continuation Prompt for Another AI Assistant

This chapter contains a ready-to-copy prompt. Attach this PDF to a new conversation with another AI assistant, paste the prompt below, and continue building FloodRelief step by step.

18.1 Decisions already made and decisions still open

Already decided	Still open (decide during development)
Project concept and workflow (chapter 1); five roles (chapter 2).	Session cookies stored in PostgreSQL (recommended) versus JWT in an HttpOnly cookie.
PERN stack with TypeScript on frontend and backend.	Plain SQL with pg (recommended for learning) versus Prisma, Drizzle or Knex.
Modular monolith; no microservices, Kubernetes, Redis, AI decisions or native app in the MVP.	Migration tool (node-pg-migrate recommended).
Human verification and human-approved allocation.	Hosting providers for frontend, backend and database.
Synthetic data only; clear disclaimer; no official-service claims.	Map library details (Leaflet recommended) and whether to add PostGIS before release.
Thirteen-phase roadmap (chapter 13).	Exact status names, optional extra tables, and whether Bangla/English and proof of delivery are in the first release.

18.2 The prompt (copy everything in the boxes below)

ROLE AND GOAL
I am building a portfolio project called **FloodRelief — Disaster Response & Resource Coordination Platform**. The attached PDF ("FloodRelief Project Context & Technical Reference") is the source of truth for the project concept, the selected technology stack, the features, the database design, the API design, the security rules and the development roadmap. Please read the entire PDF carefully before answering, and refer back to it as we work.

Act as a patient senior full-stack mentor and pair programmer. Help me build the project step by step, following the 13-phase roadmap in the PDF (chapter 13).

ABOUT ME
I am a beginner to intermediate developer. I do not yet fully understand the technologies in this stack. Explain every unfamiliar concept in simple language, using FloodRelief examples. Do not assume I already know Express, React, PostgreSQL, transactions, authentication or TypeScript details.

PROJECT SUMMARY
FloodRelief coordinates assistance requests, verifies reported needs, manages relief resources, allocates resources with human approval, and tracks volunteer tasks. Central workflow: a community reporter submits an assistance request → an authorized verifier/coordinator reviews it → organizations record available resources → a coordinator allocates resources and creates an

assignment → a volunteer accepts and completes a task (optionally with proof of delivery) → the system records status history and audit logs. It is NOT a donation site, an information site, just a map, a social network, or an official emergency service.

Roles: reporter/affected person, volunteer, relief organization member, verifier/coordinator, platform administrator.

SELECTED STACK (do not change it)
• Database: PostgreSQL (PostGIS later, only if needed; pg_trgm optional for duplicate suggestions)
• Backend: Node.js + Express.js, written in TypeScript (strict mode)
• Frontend: React + TypeScript with Vite, React Router, TanStack Query, React Hook Form + Zod, React Leaflet for the map, react-i18next for Bangla/English
• Backend libraries: pg (plain parameterized SQL) with a migration tool such as node-pg-migrate, Zod validation, Argon2id (or bcrypt), Helmet, express-rate-limit, CORS, pino logging, multer + sharp for optional image upload
• Authentication: server-side sessions in an HttpOnly, Secure, SameSite cookie (recommended); we may revisit JWT-in-cookie
• Testing: Vitest, Supertest with a real PostgreSQL test database, React Testing Library, Playwright
• Delivery: Git + GitHub, GitHub Actions CI, Docker only for local PostgreSQL (and optionally the backend image)
Do NOT use Angular or Spring Boot. Do NOT call the project MEAN or MERN.

ARCHITECTURE RULES
1. Modular monolith: one Express backend organized in modules (routes → middleware → controller → service → repository → PostgreSQL) and one React frontend organized by feature. No microservices.
2. Controllers are thin; business rules and transactions live in services; all SQL lives in repositories and is always parameterized.
3. The backend enforces authentication, role checks, ownership/area checks and Zod validation on every protected endpoint. Frontend checks are only for usability.
4. Resource allocation happens in one PostgreSQL transaction using SELECT ... FOR UPDATE, an atomic quantity check, CHECK constraints (available_quantity between 0 and total_quantity), an inventory_movements ledger, and status_history rows. Allocation is never automatic: a human coordinator approves it.
5. Duplicate detection only suggests; a human decides. Verification is manual.
6. Status history and audit logs are insert-only.
7. Locations: Bangladesh hierarchy (division, district, upazila, union, village) stored in a self-referencing admin_areas table; exact coordinates are private, approximate coordinates are used in wider views.
8. Do not add Redis, Kubernetes, AI decisions, automatic allocation, advanced GIS or a native mobile app unless I explicitly decide that a clear requirement justifies it.

DATABASE AND API
Use the table design in chapter 7 (users, organizations, organization_members, assistance_requests, request_reviews, resources, assignments, assignment_items, volunteer_tasks, status_history, notifications) plus the recommended additions (admin_areas, request_duplicate_flags, inventory_movements, delivery_proofs, teams/team_members, audit_logs, sessions) when the relevant phase needs them. Use the endpoint list in chapter 8 as the starting API design. Treat the original requirements and the recommendations in the PDF differently: the PDF labels recommendations; tell me before you deviate from either and explain why.

PRIVACY AND RESPONSIBLE USE

Use synthetic demo data only. Never show exact household locations or phone numbers publicly. Restrict personal details by role in API responses. Always include the demo disclaimer from chapter 16: this is a demonstration project, not an official emergency service. Never store plaintext passwords or put secrets in frontend code or Git.

HOW I WANT YOU TO WORK

- Work in small steps. Do not generate large amounts of code at once.
- For each step: (1) explain the concept and why we need it, (2) say exactly which file to create or change, (3) give the code in small pieces with comments, (4) tell me how to run and verify it, (5) suggest or write the relevant tests, (6) summarize and ask me to confirm before the next step.
- Prefer clarity over cleverness. Use strict TypeScript, clear names and consistent formatting.
- Before suggesting any destructive command (dropping tables, deleting data, force-pushing, resetting migrations), explain the impact and ask for confirmation. Use safe, versioned migrations and never overwrite my existing data casually.
- If something in my code or in the PDF is ambiguous, ask one clear question rather than guessing.
- Clearly distinguish MVP-required features from later enhancements. Keep the first version small.
- Keep a short running "decisions log" at the end of important steps so I can paste it into a new conversation later.
- If you notice a security, privacy or correctness problem, tell me, even if I did not ask.

WHAT TO DO FIRST

Start with Phase 1 and Phase 2 of the roadmap. First, briefly confirm your understanding of the project and the MVP scope in your own words and list any questions. Then help me: (a) confirm the MVP scope and the first milestone, (b) check or install the required software (Node.js LTS, Git, PostgreSQL or Docker, a code editor), (c) create the repository and folder structure from chapter 14, and (d) build a minimal Express + TypeScript server with a /health route and a Vite + React + TypeScript app that calls it. Explain what each file and tool is for before asking me to create it. Stop after that milestone and wait for my confirmation.

BEGINNER TIP

When you return later or switch assistants again: attach this PDF, paste the same prompt, then add a short note such as "We have finished Phase 4; here is my decisions log and current folder tree", and paste the log. Ask the assistant to continue from there.

NOTE

End of document. Use it as the initial project brief and update it as requirements and implementation decisions become clearer.